

10.1 Basic configuration

The Ethernet controller is configured using the following registers:

1. Power: In the PCONP register ([Table 46](#)), set bit PCENET.
Remark: On reset, the Ethernet block is disabled (PCENET = 0).
2. Clock: see [Table 38](#).
3. Pins: Enable Ethernet pins through the PINSEL registers and select their modes through the PINMODE registers, see [Section 8.5](#).
4. Wake-up: Activity on the Ethernet port can wake up the microcontroller from Power-down mode, see [Section 4.8.8](#).
5. Interrupts: Interrupts are enabled in the NVIC using the appropriate Interrupt Set Enable register.
6. Initialization: see [Section 10.17.2](#).

10.2 Introduction

The Ethernet block contains a full featured 10 Mbps or 100 Mbps Ethernet MAC (Media Access Controller) designed to provide optimized performance through the use of DMA hardware acceleration. Features include a generous suite of control registers, half or full duplex operation, flow control, control frames, hardware acceleration for transmit retry, receive packet filtering and wake-up on LAN activity. Automatic frame transmission and reception with Scatter-Gather DMA off-loads many operations from the CPU.

The Ethernet block is an AHB master that drives the AHB bus matrix. Through the matrix, it has access to all on-chip RAM memories. A recommended use of RAM by the Ethernet is to use one of the RAM blocks exclusively for Ethernet traffic. That RAM would then be accessed only by the Ethernet and the CPU, and possibly the GPDMA, giving maximum bandwidth to the Ethernet function.

The Ethernet block interfaces between an off-chip Ethernet PHY using the RMII (Reduced Media Independent Interface) protocol and the on-chip MIIM (Media Independent Interface Management) serial bus, also referred to as MDIO (Management Data Input/Output).

Table 125. Ethernet acronyms, abbreviations, and definitions

Acronym or Abbreviation	Definition
AHB	Advanced High-performance bus
CRC	Cyclic Redundancy Check
DMA	Direct Memory Access
Double-word	64-bit entity
FCS	Frame Check Sequence (CRC)
Fragment	A (part of an) Ethernet frame; one or multiple fragments can add up to a single Ethernet frame.

Table 125. Ethernet acronyms, abbreviations, and definitions ...continued

Acronym or Abbreviation	Definition
Frame	An Ethernet frame consists of destination address, source address, length type field, payload and frame check sequence.
Half-word	16-bit entity
LAN	Local Area Network
MAC	Media Access Control sublayer
MII	Media Independent Interface
MIIM	MII management
Octet	An 8-bit data entity, used in lieu of "byte" by IEEE 802.3
Packet	A frame that is transported across Ethernet; a packet consists of a preamble, a start of frame delimiter and an Ethernet frame.
PHY	Ethernet Physical Layer
RMII	Reduced MII
Rx	Receive
TCP/IP	Transmission Control Protocol / Internet Protocol. The most common high-level protocol used with Ethernet.
Tx	Transmit
VLAN	Virtual LAN
WoL	Wake-up on LAN
Word	32-bit entity

10.3 Features

- Ethernet standards support:
 - Supports 10 or 100 Mbps PHY devices including 10 Base-T, 100 Base-TX, 100 Base-FX, and 100 Base-T4.
 - Fully compliant with IEEE standard 802.3.
 - Fully compliant with 802.3x Full Duplex Flow Control and Half Duplex back pressure.
 - Flexible transmit and receive frame options.
 - VLAN frame support.
- Memory management:
 - Independent transmit and receive buffers memory mapped to shared SRAM.
 - DMA managers with scatter/gather DMA and arrays of frame descriptors.
 - Memory traffic optimized by buffering and prefetching.
- Enhanced Ethernet features:
 - Receive filtering.
 - Multicast and broadcast frame support for both transmit and receive.
 - Optional automatic FCS insertion (CRC) for transmit.
 - Selectable automatic transmit frame padding.

- Over-length frame support for both transmit and receive allows any length frames.
- Promiscuous receive mode.
- Automatic collision backoff and frame retransmission.
- Includes power management by clock switching.
- Wake-on-LAN power management support allows system wake-up: using the receive filters or a magic frame detection filter.
- Physical interface:
 - Attachment of external PHY chip through a standard Reduced MII (RMII) interface.
 - PHY register access is available via the Media Independent Interface Management (MIIM) interface.

10.4 Architecture and operation

Figure 18 shows the internal architecture of the Ethernet block.

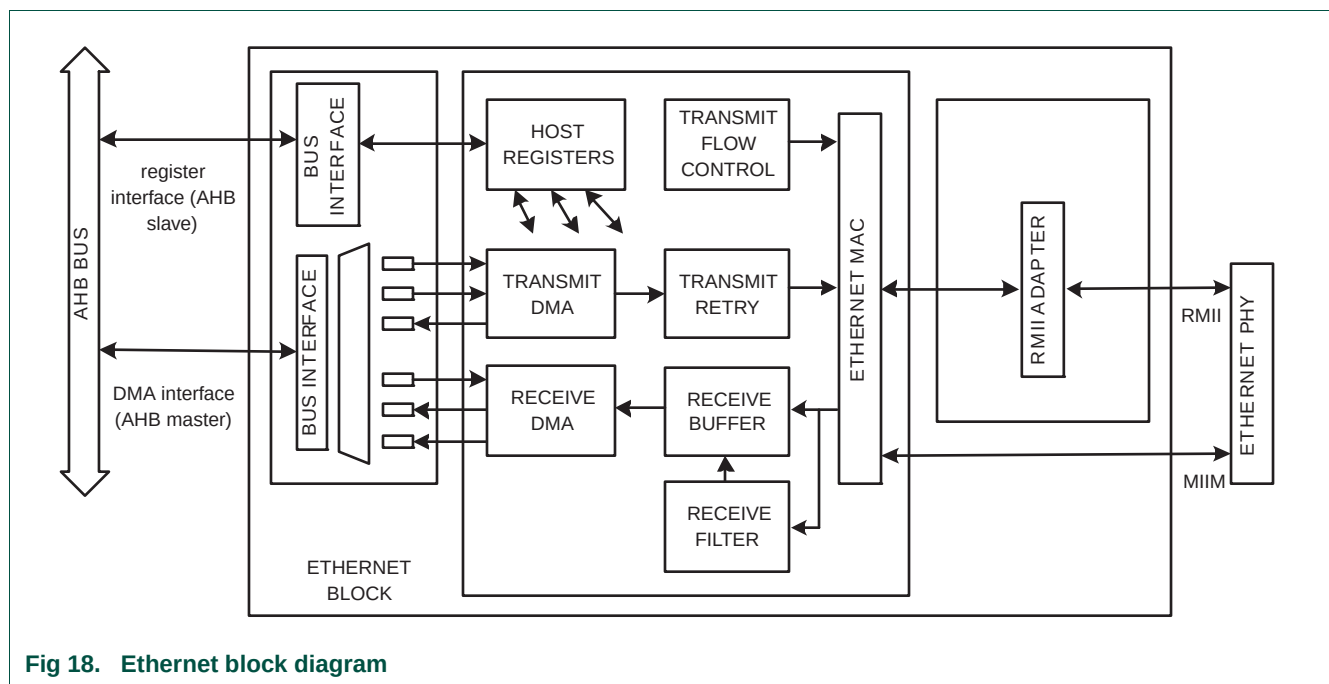


Fig 18. Ethernet block diagram

The block diagram for the Ethernet block consists of:

- The host registers module containing the registers in the software view and handling AHB accesses to the Ethernet block. The host registers connect to the transmit and receive data path as well as the MAC.
- The DMA to AHB interface. This provides an AHB master connection that allows the Ethernet block to access on-chip SRAM for reading of descriptors, writing of status, and reading and writing data buffers.
- The Ethernet MAC, which interfaces to the off-chip PHY via an RMII interface.
- The transmit data path, including:

- The transmit DMA manager which reads descriptors and data from memory and writes status to memory.
- The transmit retry module handling Ethernet retry and abort situations.
- The transmit flow control module which can insert Ethernet pause frames.
- The receive data path, including:
 - The receive DMA manager which reads descriptors from memory and writes data and status to memory.
 - The Ethernet MAC which detects frame types by parsing part of the frame header.
 - The receive filter which can filter out certain Ethernet frames by applying different filtering schemes.
 - The receive buffer implementing a delay for receive frames to allow the filter to filter out certain frames before storing them to memory.

10.5 DMA engine functions

The Ethernet block is designed to provide optimized performance via DMA hardware acceleration. Independent scatter/gather DMA engines connected to the AHB bus off-load many data transfers from the CPU.

Descriptors, which are stored in memory, contain information about fragments of incoming or outgoing Ethernet frames. A fragment may be an entire frame or a much smaller amount of data. Each descriptor contains a pointer to a memory buffer that holds data associated with a fragment, the size of the fragment buffer, and details of how the fragment will be transmitted or received.

Descriptors are stored in arrays in memory, which are located by pointer registers in the Ethernet block. Other registers determine the size of the arrays, point to the next descriptor in each array that will be used by the DMA engine, and point to the next descriptor in each array that will be used by the Ethernet device driver.

10.6 Overview of DMA operation

The DMA engine makes use of a Receive descriptor array and a Transmit descriptor array in memory. All or part of an Ethernet frame may be contained in a memory buffer associated with a descriptor. When transmitting, the transmit DMA engine uses as many descriptors as needed (one or more) to obtain (gather) all of the parts of a frame, and sends them out in sequence. When receiving, the receive DMA engine also uses as many descriptors as needed (one or more) to find places to store (scatter) all of the data in the received frame.

The base address registers for the descriptor array, registers indicating the number of descriptor array entries, and descriptor array input/output pointers are contained in the Ethernet block. The descriptor entries and all transmit and receive packet data are stored in memory which is not a part of the Ethernet block. The descriptor entries tell where related frame data is stored in memory, certain aspects of how the data is handled, and the result status of each Ethernet transaction.

Hardware in the DMA engine controls how data incoming from the Ethernet MAC is saved to memory, causes fragment related status to be saved, and advances the hardware receive pointer for incoming data. Driver software must handle the disposition of received data, changing of descriptor data addresses (to avoid unnecessary data movement), and advancing the software receive pointer. The two pointers create a circular queue in the descriptor array and allow both the DMA hardware and the driver software to know which descriptors (if any) are available for their use, including whether the descriptor array is empty or full.

Similarly, driver software must set up pointers to data that will be transmitted by the Ethernet MAC, giving instructions for each fragment of data, and advancing the software transmit pointer for outgoing data. Hardware in the DMA engine reads this information and sends the data to the Ethernet MAC interface when possible, updating the status and advancing the hardware transmit pointer.

10.7 Ethernet Packet

[Figure 19](#) illustrates the different fields in an Ethernet packet.

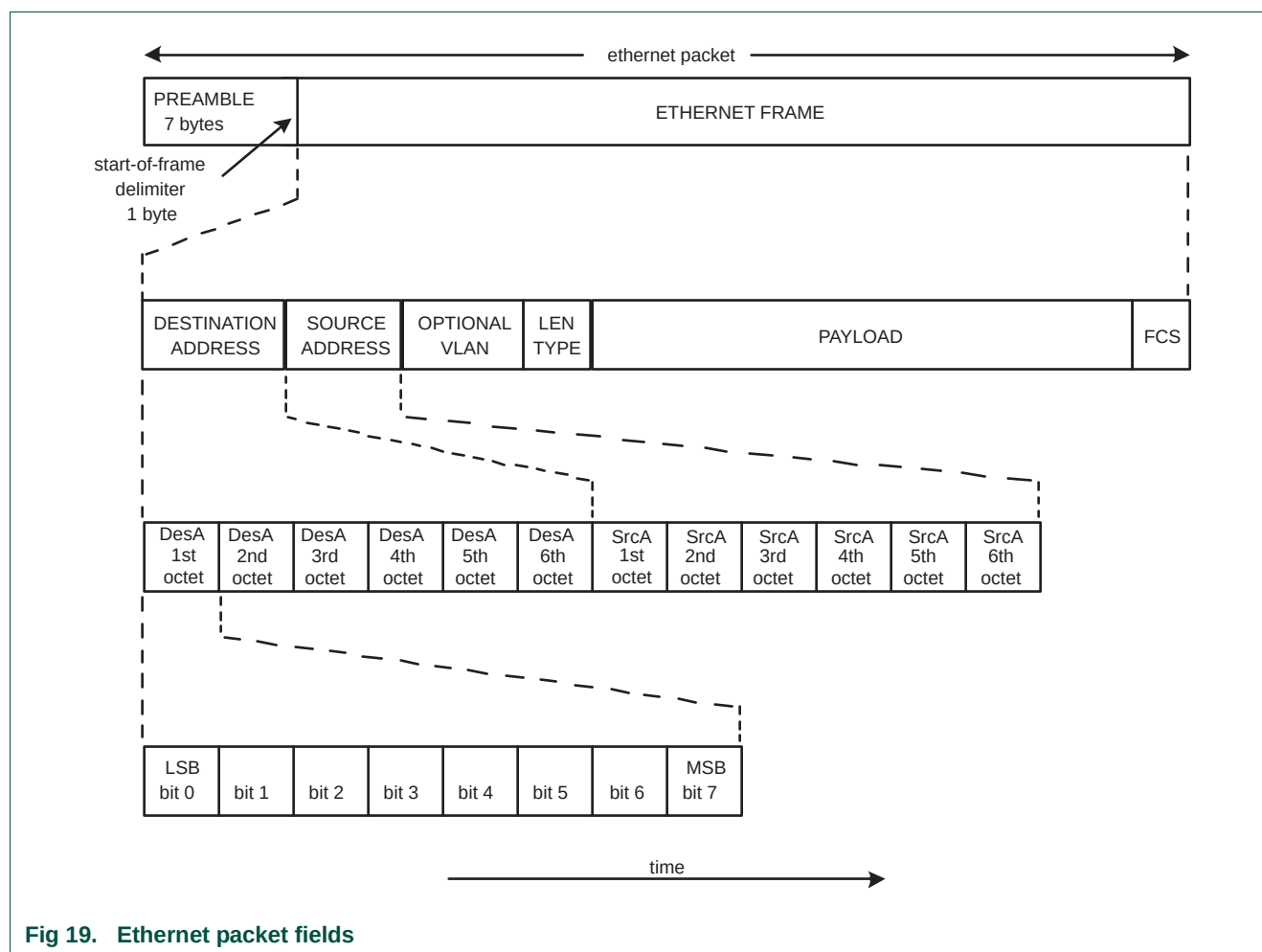


Fig 19. Ethernet packet fields

A packet consists of a preamble, a start-of-frame delimiter and an Ethernet frame.

The Ethernet frame consists of the destination address, the source address, an optional VLAN field, the length/type field, the payload and the frame check sequence.

Each address consists of 6 bytes where each byte consists of 8 bits. Bits are transferred starting with the least significant bit.

10.8 Overview

10.8.1 Partitioning

The Ethernet block and associated device driver software offer the functionality of the Media Access Control (MAC) sublayer of the data link layer in the OSI reference model (see IEEE std 802.3). The MAC sublayer offers the service of transmitting and receiving frames to the next higher protocol level, the MAC client layer, typically the Logical Link Control sublayer. The device driver software implements the interface to the MAC client layer. It sets up registers in the Ethernet block, maintains descriptor arrays pointing to frames in memory and receives results back from the Ethernet block through interrupts. When a frame is transmitted, the software partially sets up the Ethernet frames by providing pointers to the destination address field, source address field, the length/type field, the MAC client data field and optionally the CRC in the frame check sequence field. Preferably concatenation of frame fields should be done by using the scatter/gather functionality of the Ethernet core to avoid unnecessary copying of data. The hardware adds the preamble and start frame delimiter fields and can optionally add the CRC, if requested by software. When a packet is received the hardware strips the preamble and start frame delimiter and passes the rest of the packet - the Ethernet frame - to the device driver, including destination address, source address, length/type field, MAC client data and frame check sequence (FCS).

Apart from the MAC, the Ethernet block contains receive and transmit DMA managers that control receive and transmit data streams between the MAC and the AHB interface. Frames are passed via descriptor arrays located in host memory, so that the hardware can process many frames without software/CPU support. Frames can consist of multiple fragments that are accessed with scatter/gather DMA. The DMA managers optimize memory bandwidth using prefetching and buffering.

A receive filter block is used to identify received frames that are not addressed to this Ethernet station, so that they can be discarded. The Rx filters include a perfect address filter and a hash filter.

Wake-on-LAN power management support makes it possible to wake the system up from a power-down state -a state in which some of the clocks are switched off -when wake-up frames are received over the LAN. Wake-up frames are recognized by the receive filtering modules or by a Magic Frame detection technology. System wake-up occurs by triggering an interrupt.

An interrupt logic block raises and masks interrupts and keeps track of the cause of interrupts. The interrupt block sends an interrupt request signal to the host system. Interrupts can be enabled, cleared and set by software.

Support for IEEE 802.3/clause 31 flow control is implemented in the flow control block. Receive flow control frames are automatically handled by the MAC. Transmit flow control frames can be initiated by software. In half duplex mode, the flow control module will generate back pressure by sending out continuous preamble only, interrupted by pauses to prevent the jabber limit from being exceeded.

The Ethernet block has a standard Reduced Media Independent Interface (RMII) to connect to an external Ethernet PHY chip. Registers in the PHY chip are accessed via the AHB interface through the serial management connection of the MIIM bus, typically operating at 2.5 MHz.

10.8.2 Example PHY Devices

Some examples of compatible PHY devices are shown in [Table 126](#).

Table 126. Example PHY Devices

Manufacturer	Part Number(s)
Broadcom	BCM5221
ICS	ICS1893
Intel	LXT971A
LSI Logic	L80223, L80225, L80227
Micrel	KS8721
National	DP83847, DP83846, DP83843
SMSC	LAN83C185

10.9 Pin description

[Table 127](#) shows the signals used for connecting the Reduced Media Independent Interface (RMII) to the external PHY.

Table 127. Ethernet RMII pin descriptions

Pin Name	Type	Pin Description
ENET_TX_EN	Output	Transmit data enable
ENET_TXD[1:0]	Output	Transmit data, 2 bits
ENET_RXD[1:0]	Input	Receive data, 2 bits.
ENET_RX_ER	Input	Receive error.
ENET_CRD	Input	Carrier sense/data valid.
ENET_REF_CLK/ ENET_RX_CLK	Input	Reference clock

[Table 128](#) shows the signals used for Media Independent Interface Management (MIIM) of the external PHY.

Table 128. Ethernet MIIM pin descriptions

Pin Name	Type	Pin Description
ENET_MDC	Output	MIIM clock.
ENET_MDIO	Input/Output	MI data input and output

10.10 Registers and software interface

The software interface of the Ethernet block consists of a register view and the format definitions for the transmit and receive descriptors. These two aspects are addressed in the next two subsections.

10.10.1 Register map

[Table 129](#) lists the registers, register addresses and other basic information. The total AHB address space required is 4 kilobytes.

After a hard reset or a soft reset via the RegReset bit of the Command register all bits in all registers are reset to 0 unless stated otherwise in the following register descriptions.

Some registers will have unused bits which will return a 0 on a read via the AHB interface. Writing to unused register bits of an otherwise writable register will not have side effects.

The register map consists of registers in the Ethernet MAC and registers around the core for controlling DMA transfers, flow control and filtering.

Reading from reserved addresses or reserved bits leads to unpredictable data. Writing to reserved addresses or reserved bits has no effect.

Reading of write-only registers will return a read error on the AHB interface. Writing of read-only registers will return a write error on the AHB interface.

Table 129. Ethernet register definitions

Name	Description	Access	Reset Value	Address
MAC registers				
MAC1	MAC configuration register 1.	R/W	0x8000	0x5000 0000
MAC2	MAC configuration register 2.	R/W	0	0x5000 0004
IPGT	Back-to-Back Inter-Packet-Gap register.	R/W	0	0x5000 0008
IPGR	Non Back-to-Back Inter-Packet-Gap register.	R/W	0	0x5000 000C
CLRT	Collision window / Retry register.	R/W	0x370F	0x5000 0010
MAXF	Maximum Frame register.	R/W	0x0600	0x5000 0014
SUPP	PHY Support register.	R/W	0	0x5000 0018
TEST	Test register.	R/W	0	0x5000 001C
MCFG	MII Mgmt Configuration register.	R/W	0	0x5000 0020
MCMD	MII Mgmt Command register.	R/W	0	0x5000 0024
MADR	MII Mgmt Address register.	R/W	0	0x5000 0028
MWTD	MII Mgmt Write Data register.	WO	0	0x5000 002C
MRDD	MII Mgmt Read Data register.	RO	0	0x5000 0030
MIND	MII Mgmt Indicators register.	RO	0	0x5000 0034
SA0	Station Address 0 register.	R/W	0	0x5000 0040
SA1	Station Address 1 register.	R/W	0	0x5000 0044
SA2	Station Address 2 register.	R/W	0	0x5000 0048
Control registers				
Command	Command register.	R/W	0	0x5000 0100

Table 129. Ethernet register definitions ...continued

Name	Description	Access	Reset Value	Address
Status	Status register.	RO	0	0x5000 0104
RxDescriptor	Receive descriptor base address register.	R/W	0	0x5000 0108
RxStatus	Receive status base address register.	R/W	0	0x5000 010C
RxDescriptorNumber	Receive number of descriptors register.	R/W	0	0x5000 0110
RxProduceIndex	Receive produce index register.	RO	0	0x5000 0114
RxConsumeIndex	Receive consume index register.	R/W	0	0x5000 0118
TxDescriptor	Transmit descriptor base address register.	R/W	0	0x5000 011C
TxStatus	Transmit status base address register.	R/W	0	0x5000 0120
TxDescriptorNumber	Transmit number of descriptors register.	R/W	0	0x5000 0124
TxProduceIndex	Transmit produce index register.	R/W	0	0x5000 0128
TxConsumeIndex	Transmit consume index register.	RO	0	0x5000 012C
TSV0	Transmit status vector 0 register.	RO	0	0x5000 0158
TSV1	Transmit status vector 1 register.	RO	0	0x5000 015C
RSV	Receive status vector register.	RO	0	0x5000 0160
FlowControlCounter	Flow control counter register.	R/W	0	0x5000 0170
FlowControlStatus	Flow control status register.	RO	0	0x5000 0174
Rx filter registers				
RxFiliterCtrl	Receive filter control register.		0	0x5000 0200
RxFilterWoLStatus	Receive filter WoL status register.		0	0x5000 0204
RxFilterWoLClear	Receive filter WoL clear register.		0	0x5000 0208
HashFilterL	Hash filter table LSBs register.		0	0x5000 0210
HashFilterH	Hash filter table MSBs register.		0	0x5000 0214
Module control registers				
IntStatus	Interrupt status register.	RO	0	0x5000 0FE0
IntEnable	Interrupt enable register.	R/W	0	0x5000 0FE4
IntClear	Interrupt clear register.	WO	0	0x5000 0FE8
IntSet	Interrupt set register.	WO	0	0x5000 0FEC
PowerDown	Power-down register.	R/W	0	0x5000 0FF4

The third column in the table lists the accessibility of the register: read-only, write-only, read/write.

All AHB register write transactions except for accesses to the interrupt registers are posted i.e. the AHB transaction will complete before write data is actually committed to the register. Accesses to the interrupt registers will only be completed by accepting the write data when the data has been committed to the register.

10.11 Ethernet MAC register definitions

This section defines the bits in the individual registers of the Ethernet block register map.

10.11.1 MAC Configuration Register 1 (MAC1 - 0x5000 0000)

The MAC configuration register 1 (MAC1) has an address of 0x5000 0000. Its bit definition is shown in [Table 130](#).

Table 130. MAC Configuration register 1 (MAC1 - address 0x5000 0000) bit description

Bit	Symbol	Function	Reset value
0	RECEIVE ENABLE	Set this to allow receive frames to be received. Internally the MAC synchronizes this control bit to the incoming receive stream.	0
1	PASS ALL RECEIVE FRAMES	When enabled (set to '1'), the MAC will pass all frames regardless of type (normal vs. Control). When disabled, the MAC does not pass valid Control frames.	0
2	RX FLOW CONTROL	When enabled (set to '1'), the MAC acts upon received PAUSE Flow Control frames. When disabled, received PAUSE Flow Control frames are ignored.	0
3	TX FLOW CONTROL	When enabled (set to '1'), PAUSE Flow Control frames are allowed to be transmitted. When disabled, Flow Control frames are blocked.	0
4	LOOPBACK	Setting this bit will cause the MAC Transmit interface to be looped back to the MAC Receive interface. Clearing this bit results in normal operation.	0
7:5	-	Unused	0x0
8	RESET TX	Setting this bit will put the Transmit Function logic in reset.	0
9	RESET MCS / TX	Setting this bit resets the MAC Control Sublayer / Transmit logic. The MCS logic implements flow control.	0
10	RESET RX	Setting this bit will put the Ethernet receive logic in reset.	0
11	RESET MCS / RX	Setting this bit resets the MAC Control Sublayer / Receive logic. The MCS logic implements flow control.	0x0
13:12	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0
14	SIMULATION RESET	Setting this bit will cause a reset to the random number generator within the Transmit Function.	0
15	SOFT RESET	Setting this bit will put all modules within the MAC in reset except the Host Interface.	1
31:16	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0

10.11.2 MAC Configuration Register 2 (MAC2 - 0x5000 0004)

The MAC configuration register 2 (MAC2) has an address of 0x5000 0004. Its bit definition is shown in [Table 131](#).

Table 131. MAC Configuration register 2 (MAC2 - address 0x5000 0004) bit description

Bit	Symbol	Function	Reset value
0	FULL-DUPLEX	When enabled (set to '1'), the MAC operates in Full-Duplex mode. When disabled, the MAC operates in Half-Duplex mode.	0
1	FRAME LENGTH CHECKING	When enabled (set to '1'), both transmit and receive frame lengths are compared to the Length/Type field. If the Length/Type field represents a length then the check is performed. Mismatches are reported in the StatusInfo word for each received frame.	0
2	HUGE FRAME ENABLE	When enabled (set to '1'), frames of any length are transmitted and received.	0
3	DELAYED CRC	This bit determines the number of bytes, if any, of proprietary header information that exist on the front of IEEE 802.3 frames. When 1, four bytes of header (ignored by the CRC function) are added. When 0, there is no proprietary header.	0
4	CRC ENABLE	Set this bit to append a CRC to every frame whether padding was required or not. Must be set if PAD/CRC ENABLE is set. Clear this bit if frames presented to the MAC contain a CRC.	0
5	PAD / CRC ENABLE	Set this bit to have the MAC pad all short frames. Clear this bit if frames presented to the MAC have a valid length. This bit is used in conjunction with AUTO PAD ENABLE and VLAN PAD ENABLE. See Table 133 - Pad Operation for details on the pad function.	0
6	VLAN PAD ENABLE	Set this bit to cause the MAC to pad all short frames to 64 bytes and append a valid CRC. Consult Table 133 - Pad Operation for more information on the various padding features. Note: This bit is ignored if PAD / CRC ENABLE is cleared.	0
7	AUTO DETECT PAD ENABLE	Set this bit to cause the MAC to automatically detect the type of frame, either tagged or un-tagged, by comparing the two octets following the source address with 0x8100 (VLAN Protocol ID) and pad accordingly. Table 133 - Pad Operation provides a description of the pad function based on the configuration of this register. Note: This bit is ignored if PAD / CRC ENABLE is cleared.	0
8	PURE PREAMBLE ENFORCEMENT	When enabled (set to '1'), the MAC will verify the content of the preamble to ensure it contains 0x55 and is error-free. A packet with an incorrect preamble is discarded. When disabled, no preamble checking is performed.	0
9	LONG PREAMBLE ENFORCEMENT	When enabled (set to '1'), the MAC only allows receive packets which contain preamble fields less than 12 bytes in length. When disabled, the MAC allows any length preamble as per the Standard.	0
11:10	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0
12	NO BACKOFF	When enabled (set to '1'), the MAC will immediately retransmit following a collision rather than using the Binary Exponential Backoff algorithm as specified in the Standard.	0
13	BACK PRESSURE / NO BACKOFF	When enabled (set to '1'), after the MAC incidentally causes a collision during back pressure, it will immediately retransmit without backoff, reducing the chance of further collisions and ensuring transmit packets get sent.	0
14	EXCESS DEFER	When enabled (set to '1') the MAC will defer to carrier indefinitely as per the Standard. When disabled, the MAC will abort when the excessive deferral limit is reached.	0
31:15	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0

Table 132. Pad operation

Type	Auto detect pad enable MAC2 [7]	VLAN pad enable MAC2 [6]	Pad/CRC enable MAC2 [5]	Action
Any	x	x	0	No pad or CRC check
Any	0	0	1	Pad to 60 bytes, append CRC
Any	x	1	1	Pad to 64 bytes, append CRC
Any	1	0	1	If untagged, pad to 60 bytes and append CRC. If VLAN tagged: pad to 64 bytes and append CRC.

10.11.3 Back-to-Back Inter-Packet-Gap Register (IPGT - 0x5000 0008)

The Back-to-Back Inter-Packet-Gap register (IPGT) has an address of 0x5000 0008. Its bit definition is shown in [Table 133](#).

Table 133. Back-to-back Inter-packet-gap register (IPGT - address 0x5000 0008) bit description

Bit	Symbol	Function	Reset value
6:0	BACK-TO-BACK INTER-PACKET-GAP	This is a programmable field representing the nibble time offset of the minimum possible period between the end of any transmitted packet to the beginning of the next. In Full-Duplex mode, the register value should be the desired period in nibble times minus 3. In Half-Duplex mode, the register value should be the desired period in nibble times minus 6. In Full-Duplex the recommended setting is 0x15 (21d), which represents the minimum IPG of 960 ns (in 100 Mbps mode) or 9.6 μ s (in 10 Mbps mode). In Half-Duplex the recommended setting is 0x12 (18d), which also represents the minimum IPG of 960 ns (in 100 Mbps mode) or 9.6 μ s (in 10 Mbps mode).	0x0
31:7	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0

10.11.4 Non Back-to-Back Inter-Packet-Gap Register (IPGR - 0x5000 000C)

The Non Back-to-Back Inter-Packet-Gap register (IPGR) has an address of 0x5000 000C. Its bit definition is shown in [Table 134](#).

Table 134. Non Back-to-back Inter-packet-gap register (IPGR - address 0x5000 000C) bit description

Bit	Symbol	Function	Reset value
6:0	NON-BACK-TO-BACK INTER-PACKET-GAP PART2	This is a programmable field representing the Non-Back-to-Back Inter-Packet-Gap. The recommended value is 0x12 (18d), which represents the minimum IPG of 960 ns (in 100 Mbps mode) or 9.6 μ s (in 10 Mbps mode).	0x0
7	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0
14:8	NON-BACK-TO-BACK INTER-PACKET-GAP PART1	This is a programmable field representing the optional carrierSense window referenced in IEEE 802.3/4.2.3.2.1 'Carrier Deference'. If carrier is detected during the timing of IPGR1, the MAC defers to carrier. If, however, carrier becomes active after IPGR1, the MAC continues timing IPGR2 and transmits, knowingly causing a collision, thus ensuring fair access to medium. Its range of values is 0x0 to IPGR2. The recommended value is 0xC (12d)	0x0
31:15	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0

10.11.5 Collision Window / Retry Register (CLRT - 0x5000 0010)

The Collision window / Retry register (CLRT) has an address of 0x5000 0010. Its bit definition is shown in [Table 135](#).

Table 135. Collision Window / Retry register (CLRT - address 0x5000 0010) bit description

Bit	Symbol	Function	Reset value
3:0	RETRANSMISSION MAXIMUM	This is a programmable field specifying the number of retransmission attempts following a collision before aborting the packet due to excessive collisions. The Standard specifies the attemptLimit to be 0xF (15d). See IEEE 802.3/4.2.3.2.5.	0xF
7:4	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	0x0
13:8	COLLISION WINDOW	This is a programmable field representing the slot time or collision window during which collisions occur in properly configured networks. The default value of 0x37 (55d) represents a 56 byte window following the preamble and SFD.	0x37
31:14	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

10.11.6 Maximum Frame Register (MAXF - 0x5000 0014)

The Maximum Frame register (MAXF) has an address of 0x5000 0014. Its bit definition is shown in [Table 136](#).

Table 136. Maximum Frame register (MAXF - address 0x5000 0014) bit description

Bit	Symbol	Function	Reset value
15:0	MAXIMUM FRAME LENGTH	This field resets to the value 0x0600, which represents a maximum receive frame of 1536 octets. An untagged maximum size Ethernet frame is 1518 octets. A tagged frame adds four octets for a total of 1522 octets. If a shorter maximum length restriction is desired, program this 16-bit field.	0x0600
31:16	-	Unused	0x0

10.11.7 PHY Support Register (SUPP - 0x5000 0018)

The PHY Support register (SUPP) has an address of 0x5000 0018. The SUPP register provides additional control over the RMII interface. The bit definition of this register is shown in [Table 137](#).

Table 137. PHY Support register (SUPP - address 0x5000 0018) bit description

Bit	Symbol	Function	Reset value
7:0	-	Unused	0x0
8	SPEED	This bit configures the Reduced MII logic for the current operating speed. When set, 100 Mbps mode is selected. When cleared, 10 Mbps mode is selected.	0
31:9	-	Unused	0x0

Unused bits in the PHY support register should be left as zeroes.

10.11.8 Test Register (TEST - 0x5000 001C)

The Test register (TEST) has an address of 0x5000 001C. The bit definition of this register is shown in [Table 138](#). These bits are used for testing purposes only.

Table 138. Test register (TEST - address 0x5000) bit description

Bit	Symbol	Function	Reset value
0	SHORTCUT PAUSE QUANTA	This bit reduces the effective PAUSE quanta from 64 byte-times to 1 byte-time.	0
1	TEST PAUSE	This bit causes the MAC Control sublayer to inhibit transmissions, just as if a PAUSE Receive Control frame with a nonzero pause time parameter was received.	0
2	TEST BACKPRESSURE	Setting this bit will cause the MAC to assert backpressure on the link. Backpressure causes preamble to be transmitted, raising carrier sense. A transmit packet from the system will be sent during backpressure.	0
31:3	-	Unused	0x0

10.11.9 MII Mgmt Configuration Register (MCFG - 0x5000 0020)

The MII Mgmt Configuration register (MCFG) has an address of 0x5000 0020. The bit definition of this register is shown in [Table 139](#).

Table 139. MII Mgmt Configuration register (MCFG - address 0x5000 0020) bit description

Bit	Symbol	Function	Reset value
0	SCAN INCREMENT	Set this bit to cause the MII Management hardware to perform read cycles across a range of PHYs. When set, the MII Management hardware will perform read cycles from address 1 through the value set in PHY ADDRESS[4:0]. Clear this bit to allow continuous reads of the same PHY.	0
1	SUPPRESS PREAMBLE	Set this bit to cause the MII Management hardware to perform read/write cycles without the 32-bit preamble field. Clear this bit to cause normal cycles to be performed. Some PHYs support suppressed preamble.	0
5:2	CLOCK SELECT	This field is used by the clock divide logic in creating the MII Management Clock (MDC) which IEEE 802.3u defines to be no faster than 2.5 MHz. Some PHYs support clock rates up to 12.5 MHz, however. The AHB bus clock (HCLK) is divided by the specified amount. Refer to Table 140 below for the definition of values for this field.	0
14:6	-	Unused	0x0
15	RESET MII MGMT	This bit resets the MII Management hardware.	0
31:16	-	Unused	0x0

Table 140. Clock select encoding

Clock Select	Bit 5	Bit 4	Bit 3	Bit 2	Maximum AHB clock supported
Host Clock divided by 4	0	0	0	x	10
Host Clock divided by 6	0	0	1	0	15
Host Clock divided by 8	0	0	1	1	20
Host Clock divided by 10	0	1	0	0	25
Host Clock divided by 14	0	1	0	1	35
Host Clock divided by 20	0	1	1	0	50
Host Clock divided by 28	0	1	1	1	70
Host Clock divided by 36	1	0	0	0	80 ^[1]
Host Clock divided by 40	1	0	0	1	90 ^[1]
Host Clock divided by 44	1	0	1	0	100 ^[1]

Table 140. Clock select encoding

Clock Select	Bit 5	Bit 4	Bit 3	Bit 2	Maximum AHB clock supported
Host Clock divided by 48	1	0	1	1	120 ^[1]
Host Clock divided by 52	1	1	0	0	130 ^[1]
Host Clock divided by 56	1	1	0	1	140 ^[1]
Host Clock divided by 60	1	1	1	0	150 ^[1]
Host Clock divided by 64	1	1	1	1	160 ^[1]

[1] The maximum AHB clock rate allowed is limited to the maximum CPU clock rate for the device.

10.11.10 MII Mgmt Command Register (MCMD - 0x5000 0024)

The MII Mgmt Command register (MCMD) has an address of 0x5000 0024. The bit definition of this register is shown in [Table 141](#).

Table 141. MII Mgmt Command register (MCMD - address 0x5000 0024) bit description

Bit	Symbol	Function	Reset value
0	READ	This bit causes the MII Management hardware to perform a single Read cycle. The Read data is returned in Register MRDD (MII Mgmt Read Data).	0
1	SCAN	This bit causes the MII Management hardware to perform Read cycles continuously. This is useful for monitoring Link Fail for example.	0
31:2	-	Unused	0x0

10.11.11 MII Mgmt Address Register (MADR - 0x5000 0028)

The MII Mgmt Address register (MADR) has an address of 0x5000 0028. The bit definition of this register is shown in [Table 142](#).

Table 142. MII Mgmt Address register (MADR - address 0x5000 0028) bit description

Bit	Symbol	Function	Reset value
4:0	REGISTER ADDRESS	This field represents the 5-bit Register Address field of Mgmt cycles. Up to 32 registers can be accessed.	0x0
7:5	-	Unused	0x0
12:8	PHY ADDRESS	This field represents the 5-bit PHY Address field of Mgmt cycles. Up to 31 PHYs can be addressed (0 is reserved).	0x0
31:13	-	Unused	0x0

10.11.12 MII Mgmt Write Data Register (MWTD - 0x5000 002C)

The MII Mgmt Write Data register (MWTD) is a write-only register with an address of 0x5000 002C. The bit definition of this register is shown in [Table 143](#).

Table 143. MII Mgmt Write Data register (MWTD - address 0x5000 002C) bit description

Bit	Symbol	Function	Reset value
15:0	WRITE DATA	When written, an MII Mgmt write cycle is performed using the 16-bit data and the pre-configured PHY and Register addresses from the MII Mgmt Address register (MADR).	0x0
31:16	-	Unused	0x0

10.11.13 MII Mgmt Read Data Register (MRDD - 0x5000 0030)

The MII Mgmt Read Data register (MRDD) is a read-only register with an address of 0x5000 0030. The bit definition of this register is shown in [Table 144](#).

Table 144. MII Mgmt Read Data register (MRDD - address 0x5000 0030) bit description

Bit	Symbol	Function	Reset value
15:0	READ DATA	Following an MII Mgmt Read Cycle, the 16-bit data can be read from this location.	0x0
31:16	-	Unused	0x0

10.11.14 MII Mgmt Indicators Register (MIND - 0x5000 0034)

The MII Mgmt Indicators register (MIND) is a read-only register with an address of 0x5000 0034. The bit definition of this register is shown in [Table 145](#).

Table 145. MII Mgmt Indicators register (MIND - address 0x5000 0034) bit description

Bit	Symbol	Function	Reset value
0	BUSY	When '1' is returned - indicates MII Mgmt is currently performing an MII Mgmt Read or Write cycle.	0
1	SCANNING	When '1' is returned - indicates a scan operation (continuous MII Mgmt Read cycles) is in progress.	0
2	NOT VALID	When '1' is returned - indicates MII Mgmt Read cycle has not completed and the Read Data is not yet valid.	0
3	MII Link Fail	When '1' is returned - indicates that an MII Mgmt link fail has occurred.	0
31:4	-	Unused	0x0

Here are two examples to access PHY via the MII Management Controller.

For PHY Write if scan is not used:

1. Write 0 to MCMD
2. Write PHY address and register address to MADR
3. Write data to MWTD
4. Wait for busy bit to be cleared in MIND

For PHY Read if scan is not used:

1. Write 1 to MCMD
2. Write PHY address and register address to MADR

3. Wait for busy bit to be cleared in MIND
4. Write 0 to MCMD
5. Read data from MRDD

10.11.15 Station Address 0 Register (SA0 - 0x5000 0040)

The Station Address 0 register (SA0) has an address of 0x5000 0040. The bit definition of this register is shown in [Table 146](#).

Table 146. Station Address register (SA0 - address 0x5000 0040) bit description

Bit	Symbol	Function	Reset value
7:0	STATION ADDRESS, 2nd octet	This field holds the second octet of the station address.	0x0
15:8	STATION ADDRESS, 1st octet	This field holds the first octet of the station address.	0x0
31:16	-	Unused	0x0

The station address is used for perfect address filtering and for sending pause control frames. For the ordering of the octets in the packet please refer to [Figure 19](#).

10.11.16 Station Address 1 Register (SA1 - 0x5000 0044)

The Station Address 1 register (SA1) has an address of 0x5000 0044. The bit definition of this register is shown in [Table 147](#).

Table 147. Station Address register (SA1 - address 0x5000 0044) bit description

Bit	Symbol	Function	Reset value
7:0	STATION ADDRESS, 4th octet	This field holds the fourth octet of the station address.	0x0
15:8	STATION ADDRESS, 3rd octet	This field holds the third octet of the station address.	0x0
31:16	-	Unused	0x0

The station address is used for perfect address filtering and for sending pause control frames. For the ordering of the octets in the packet please refer to [Figure 19](#).

10.11.17 Station Address 2 Register (SA2 - 0x5000 0048)

The Station Address 2 register (SA2) has an address of 0x5000 0048. The bit definition of this register is shown in [Table 148](#).

Table 148. Station Address register (SA2 - address 0x5000 0048) bit description

Bit	Symbol	Function	Reset value
7:0	STATION ADDRESS, 6th octet	This field holds the sixth octet of the station address.	0x0
15:8	STATION ADDRESS, 5th octet	This field holds the fifth octet of the station address.	0x0
31:16	-	Unused	0x0

The station address is used for perfect address filtering and for sending pause control frames. For the ordering of the octets in the packet please refer to [Figure 19](#).

10.12 Control register definitions

10.12.1 Command Register (Command - 0x5000 0100)

The Command register (Command) register has an address of 0x5000 0100. Its bit definition is shown in [Table 149](#).

Table 149. Command register (Command - address 0x5000 0100) bit description

Bit	Symbol	Function	Reset value
0	RxEnable	Enable receive.	0
1	TxEnable	Enable transmit.	0
2	-	Unused	0x0
3	RegReset	When a '1' is written, all datapaths and the host registers are reset. The MAC needs to be reset separately.	0
4	TxReset	When a '1' is written, the transmit datapath is reset.	0
5	RxReset	When a '1' is written, the receive datapath is reset.	0
6	PassRuntFrame	When set to '1', passes runt frames smaller than 64 bytes to memory unless they have a CRC error. If '0' runt frames are filtered out.	0
7	PassRxFilter	When set to '1', disables receive filtering i.e. all frames received are written to memory.	0
8	TxFlowControl	Enable IEEE 802.3 / clause 31 flow control sending pause frames in full duplex and continuous preamble in half duplex.	0
9	RMII	When set to "1", RMII mode is selected. This bit must be set to one during Ethernet initialization. See Section 10.17.2 .	0
10	FullDuplex	When set to "1", indicates full duplex operation.	0
31:11	-	Unused	0x0

All bits can be written and read. The Tx/RxReset bits are write-only, reading will return a 0.

10.12.2 Status Register (Status - 0x5000 0104)

The Status register (Status) is a read-only register with an address of 0x5000 0104. Its bit definition is shown in [Table 150](#).

Table 150. Status register (Status - address 0x5000 0104) bit description

Bit	Symbol	Function	Reset value
0	RxStatus	If 1, the receive channel is active. If 0, the receive channel is inactive.	0
1	TxStatus	If 1, the transmit channel is active. If 0, the transmit channel is inactive.	0
31:2	-	Unused	0x0

The values represent the status of the two channels/data paths. When the status is 1, the channel is active, meaning:

- It is enabled and the Rx/TxEnable bit is set in the Command register or it just got disabled while still transmitting or receiving a frame.
- Also, for the transmit channel, the transmit queue is not empty
i.e. ProduceIndex != ConsumeIndex.
- Also, for the receive channel, the receive queue is not full
i.e. ProduceIndex != ConsumeIndex - 1.

The status transitions from active to inactive if the channel is disabled by a software reset of the Rx/TxEnable bit in the Command register and the channel has committed the status and data of the current frame to memory. The status also transitions to inactive if the transmit queue is empty or if the receive queue is full and status and data have been committed to memory.

10.12.3 Receive Descriptor Base Address Register (RxDescriptor - 0x5000 0108)

The Receive Descriptor base address register (RxDescriptor) has an address of 0x5000 0108. Its bit definition is shown in [Table 151](#).

Table 151. Receive Descriptor Base Address register (RxDescriptor - address 0x5000 0108) bit description

Bit	Symbol	Function	Reset value
1:0	-	Fixed to '00'	-
31:2	RxDescriptor	MSBs of receive descriptor base address.	0x0

The receive descriptor base address is a byte address aligned to a word boundary i.e. LSB 1:0 are fixed to "00". The register contains the lowest address in the array of descriptors.

10.12.4 Receive Status Base Address Register (RxStatus - 0x5000 010C)

The receive descriptor base address is a byte address aligned to a word boundary i.e. LSB 1:0 are fixed to "00". The register contains the lowest address in the array of descriptors.

Table 152. receive Status Base Address register (RxStatus - address 0x5000 010C) bit description

Bit	Symbol	Function	Reset value
2:0	-	Fixed to '000'	-
31:3	RxStatus	MSBs of receive status base address.	0x0

The receive status base address is a byte address aligned to a double word boundary i.e. LSB 2:0 are fixed to "000".

10.12.5 Receive Number of Descriptors Register (RxDescriptor - 0x5000 0110)

The Receive Number of Descriptors register (RxDescriptorNumber) has an address of 0x5000 0110. Its bit definition is shown in [Table 153](#).

Table 153. Receive Number of Descriptors register (RxDescriptor - address 0x5000 0110) bit description

Bit	Symbol	Function	Reset value
15:0	RxDescriptorNumber	Number of descriptors in the descriptor array for which RxDescriptor is the base address. The number of descriptors is minus one encoded.	0x0
31:16	-	Unused	0x0

The receive number of descriptors register defines the number of descriptors in the descriptor array for which RxDescriptor is the base address. The number of descriptors should match the number of statuses. The register uses minus one encoding i.e. if the array has 8 elements, the value in the register should be 7.

10.12.6 Receive Produce Index Register (RxProduceIndex - 0x5000 0114)

The Receive Produce Index register (RxProduceIndex) is a read-only register with an address of 0x5000 0114. Its bit definition is shown in [Table 154](#).

Table 154. Receive Produce Index register (RxProduceIndex - address 0x5000 0114) bit description

Bit	Symbol	Function	Reset value
15:0	RxProduceIndex	Index of the descriptor that is going to be filled next by the receive datapath.	0x0
31:16	-	Unused	0x0

The receive produce index register defines the descriptor that is going to be filled next by the hardware receive process. After a frame has been received, hardware increments the index. The value is wrapped to 0 once the value of RxDescriptorNumber has been reached. If the RxProduceIndex equals RxConsumeIndex - 1, the array is full and any further frames being received will cause a buffer overrun error.

10.12.7 Receive Consume Index Register (RxConsumeIndex - 0x5000 0118)

The Receive consume index register (RxConsumeIndex) has an address of 0x5000 0118. Its bit definition is shown in [Table 155](#).

Table 155. Receive Consume Index register (RxConsumeIndex - address 0x5000 0118) bit description

Bit	Symbol	Function	Reset value
15:0	RxConsumeIndex	Index of the descriptor that is going to be processed next by the receive	
31:16	-	Unused	0x0

The receive consume register defines the descriptor that is going to be processed next by the software receive driver. The receive array is empty as long as RxProduceIndex equals RxConsumeIndex. As soon as the array is not empty, software can process the frame pointed to by RxConsumeIndex. After a frame has been processed by software, software should increment the RxConsumeIndex. The value must be wrapped to 0 once the value

of RxDescriptorNumber has been reached. If the RxProduceIndex equals RxConsumeIndex - 1, the array is full and any further frames being received will cause a buffer overrun error.

10.12.8 Transmit Descriptor Base Address Register (TxDescriptor - 0x5000 011C)

The Transmit Descriptor base address register (TxDescriptor) has an address of 0x5000 011C. Its bit definition is shown in [Table 156](#).

Table 156. Transmit Descriptor Base Address register (TxDescriptor - address 0x5000 011C) bit description

Bit	Symbol	Function	Reset value
1:0	-	Fixed to '00'	-
31:2	TxDescriptor	MSBs of transmit descriptor base address.	0x0

The transmit descriptor base address is a byte address aligned to a word boundary i.e. LSB 1:0 are fixed to "00". The register contains the lowest address in the array of descriptors.

10.12.9 Transmit Status Base Address Register (TxStatus - 0x5000 0120)

The Transmit Status base address register (TxStatus) has an address of 0x5000 0120. Its bit definition is shown in [Table 157](#).

Table 157. Transmit Status Base Address register (TxStatus - address 0x5000 0120) bit description

Bit	Symbol	Function	Reset value
1:0	-	Fixed to '00'	-
31:2	TxStatus	MSBs of transmit status base address.	0x0

The transmit status base address is a byte address aligned to a word boundary i.e. LSB 1:0 are fixed to "00". The register contains the lowest address in the array of statuses.

10.12.10 Transmit Number of Descriptors Register (TxDescriptorNumber - 0x5000 0124)

The Transmit Number of Descriptors register (TxDescriptorNumber) has an address of 0x5000 0124. Its bit definition is shown in [Table 158](#).

Table 158. Transmit Number of Descriptors register (TxDescriptorNumber - address 0x5000 0124) bit description

Bit	Symbol	Function	Reset value
15:0	TxDescriptorNumber	Number of descriptors in the descriptor array for which TxDescriptor is the base address. The register is minus one encoded.	
31:16	-	Unused	0x0

The transmit number of descriptors register defines the number of descriptors in the descriptor array for which TxDescriptor is the base address. The number of descriptors should match the number of statuses. The register uses minus one encoding i.e. if the array has 8 elements, the value in the register should be 7.

10.12.11 Transmit Produce Index Register (TxProduceIndex - 0x5000 0128)

The Transmit Produce Index register (TxProduceIndex) has an address of 0x5000 0128. Its bit definition is shown in [Table 159](#).

Table 159. Transmit Produce Index register (TxProduceIndex - address 0x5000 0128) bit description

Bit	Symbol	Function	Reset value
15:0	TxProduceIndex	Index of the descriptor that is going to be filled next by the transmit software driver.	0x0
31:16	-	Unused	0x0

The transmit produce index register defines the descriptor that is going to be filled next by the software transmit driver. The transmit descriptor array is empty as long as TxProduceIndex equals TxConsumeIndex. If the transmit hardware is enabled, it will start transmitting frames as soon as the descriptor array is not empty. After a frame has been processed by software, it should increment the TxProduceIndex. The value must be wrapped to 0 once the value of TxDescriptorNumber has been reached. If the TxProduceIndex equals TxConsumeIndex - 1 the descriptor array is full and software should stop producing new descriptors until hardware has transmitted some frames and updated the TxConsumeIndex.

10.12.12 Transmit Consume Index Register (TxConsumeIndex - 0x5000 012C)

The Transmit Consume Index register (TxConsumeIndex) is a read-only register with an address of 0x5000 012C. Its bit definition is shown in [Table 160](#).

Table 160. Transmit Consume Index register (TxConsumeIndex - address 0x5000 012C) bit description

Bit	Symbol	Function	Reset value
15:0	TxConsumeIndex	Index of the descriptor that is going to be transmitted next by the transmit datapath.	0x0
31:16	-	Unused	0x0

The transmit consume index register defines the descriptor that is going to be transmitted next by the hardware transmit process. After a frame has been transmitted hardware increments the index, wrapping the value to 0 once the value of TxDescriptorNumber has been reached. If the TxConsumeIndex equals TxProduceIndex the descriptor array is empty and the transmit channel will stop transmitting until software produces new descriptors.

10.12.13 Transmit Status Vector 0 Register (TSV0 - 0x5000 0158)

The Transmit Status Vector 0 register (TSV0) is a read-only register with an address of 0x5000 0158. The transmit status vector registers store the most recent transmit status returned by the MAC. Since the status vector consists of more than 4 bytes, status is

distributed over two registers TSV0 and TSV1. These registers are provided for debug purposes, because the communication between driver software and the Ethernet block takes place primarily through the frame descriptors. The status register contents are valid as long as the internal status of the MAC is valid and should typically only be read when the transmit and receive processes are halted.

[Table 161](#) lists the bit definitions of the TSV0 register.

Table 161. Transmit Status Vector 0 register (TSV0 - address 0x5000 0158) bit description

Bit	Symbol	Function	Reset value
0	CRC error	The attached CRC in the packet did not match the internally generated CRC.	0
1	Length check error	Indicates the frame length field does not match the actual number of data items and is not a type field.	0
2	Length out of range ^[1]	Indicates that frame type/length field was larger than 1500 bytes.	0
3	Done	Transmission of packet was completed.	0
4	Multicast	Packet's destination was a multicast address.	0
5	Broadcast	Packet's destination was a broadcast address.	0
6	Packet Defer	Packet was deferred for at least one attempt, but less than an excessive defer.	0
7	Excessive Defer	Packet was deferred in excess of 6071 nibble times in 100 Mbps or 24287 bit times in 10 Mbps mode.	0
8	Excessive Collision	Packet was aborted due to exceeding of maximum allowed number of collisions.	0
9	Late Collision	Collision occurred beyond collision window, 512 bit times.	0
10	Giant	Byte count in frame was greater than can be represented in the transmit byte count field in TSV1.	0
11	Underrun	Host side caused buffer underrun.	0
27:12	Total bytes	The total number of bytes transferred including collided attempts.	0x0
28	Control frame	The frame was a control frame.	0
29	Pause	The frame was a control frame with a valid PAUSE opcode.	0
30	Backpressure	Carrier-sense method backpressure was previously applied.	0
31	VLAN	Frame's length/type field contained 0x8100 which is the VLAN protocol identifier.	0

- [1] The EMAC doesn't distinguish the frame type and frame length, so, e.g. when the IP(0x8000) or ARP(0x0806) packets are received, it compares the frame type with the max length and gives the "Length out of range" error. In fact, this bit is not an error indication, but simply a statement by the chip regarding the status of the received frame.

10.12.14 Transmit Status Vector 1 Register (TSV1 - 0x5000 015C)

The Transmit Status Vector 1 register (TSV1) is a read-only register with an address of 0x5000 015C. The transmit status vector registers store the most recent transmit status returned by the MAC. Since the status vector consists of more than 4 bytes, status is distributed over two registers TSV0 and TSV1. These registers are provided for debug

purposes, because the communication between driver software and the Ethernet block takes place primarily through the frame descriptors. The status register contents are valid as long as the internal status of the MAC is valid and should typically only be read when the transmit and receive processes are halted. [Table 162](#) lists the bit definitions of the TSV1 register.

Table 162. Transmit Status Vector 1 register (TSV1 - address 0x5000 015C) bit description

Bit	Symbol	Function	Reset value
15:0	Transmit byte count	The total number of bytes in the frame, not counting the collided bytes.	0x0
19:16	Transmit collision count	Number of collisions the current packet incurred during transmission attempts. The maximum number of collisions (16) cannot be represented.	0x0
31:20	-	Unused	0x0

10.12.15 Receive Status Vector Register (RSV - 0x5000 0160)

The Receive status vector register (RSV) is a read-only register with an address of 0x5000 0160. The receive status vector register stores the most recent receive status returned by the MAC. This register is provided for debug purposes, because the communication between driver software and the Ethernet block takes place primarily through the frame descriptors. The status register contents are valid as long as the internal status of the MAC is valid and should typically only be read when the transmit and receive processes are halted.

[Table 163](#) lists the bit definitions of the RSV register.

Table 163. Receive Status Vector register (RSV - address 0x5000 0160) bit description

Bit	Symbol	Function	Reset value
15:0	Received byte count	Indicates length of received frame.	0x0
16	Packet previously ignored	Indicates that a packet was dropped.	0
17	RXDV event previously seen	Indicates that the last receive event seen was not long enough to be a valid packet.	0
18	Carrier event previously seen	Indicates that at some time since the last receive statistics, a carrier event was detected.	0
19	Receive code violation	Indicates that received PHY data does not represent a valid receive code.	0
20	CRC error	The attached CRC in the packet did not match the internally generated CRC.	0
21	Length check error	Indicates the frame length field does not match the actual number of data items and is not a type field.	0
22	Length out of range ^[1]	Indicates that frame type/length field was larger than 1518 bytes.	0
23	Receive OK	The packet had valid CRC and no symbol errors.	0
24	Multicast	The packet destination was a multicast address.	0
25	Broadcast	The packet destination was a broadcast address.	0

Table 163. Receive Status Vector register (RSV - address 0x5000 0160) bit description ...continued

Bit	Symbol	Function	Reset value
26	Dribble Nibble	Indicates that after the end of packet another 1-7 bits were received. A single nibble, called dribble nibble, is formed but not sent out.	0
27	Control frame	The frame was a control frame.	0
28	PAUSE	The frame was a control frame with a valid PAUSE opcode.	0
29	Unsupported Opcode	The current frame was recognized as a Control Frame but contains an unknown opcode.	0
30	VLAN	Frame's length/type field contained 0x8100 which is the VLAN protocol identifier.	0
31	-	Unused	0x0

- [1] The EMAC doesn't distinguish the frame type and frame length, so, e.g. when the IP(0x8000) or ARP(0x0806) packets are received, it compares the frame type with the max length and gives the "Length out of range" error. In fact, this bit is not an error indication, but simply a statement by the chip regarding the status of the received frame.

10.12.16 Flow Control Counter Register (FlowControlCounter - 0x5000 0170)

The Flow Control Counter register (FlowControlCounter) has an address of 0x5000 0170. [Table 164](#) lists the bit definitions of the register.

Table 164. Flow Control Counter register (FlowControlCounter - address 0x5000 0170) bit description

Bit	Symbol	Function	Reset value
15:0	MirrorCounter	In full duplex mode the MirrorCounter specifies the number of cycles before re-issuing the Pause control frame.	0x0
31:16	PauseTimer	In full-duplex mode the PauseTimer specifies the value that is inserted into the pause timer field of a pause flow control frame. In half duplex mode the PauseTimer specifies the number of backpressure cycles.	0x0

10.12.17 Flow Control Status Register (FlowControlStatus - 0x5000 0174)

The Flow Control Status register (FlowControlStatus) is a read-only register with an address of 0x5000 0174. [Table 165](#) lists the bit definitions of the register.

Table 165. Flow Control Status register (FlowControlStatus - address 0x5000 0174) bit description

Bit	Symbol	Function	Reset value
15:0	MirrorCounterCurrent	In full duplex mode this register represents the current value of the datapath's mirror counter which counts up to the value specified by the MirrorCounter field in the FlowControlCounter register. In half duplex mode the register counts until it reaches the value of the PauseTimer bits in the FlowControlCounter register.	0x0
31:16	-	Unused	0x0

10.13 Receive filter register definitions

10.13.1 Receive Filter Control Register (RxFilterCtrl - 0x5000 0200)

The Receive Filter Control register (RxFilterCtrl) has an address of 0x5000 0200. [Table 166](#) lists the definition of the individual bits in the register.

Table 166. Receive Filter Control register (RxFilterCtrl - address 0x5000 0200) bit description

Bit	Symbol	Function	Reset value
0	AcceptUnicastEn	When set to '1', all unicast frames are accepted.	0
1	AcceptBroadcastEn	When set to '1', all broadcast frames are accepted.	0
2	AcceptMulticastEn	When set to '1', all multicast frames are accepted.	0
3	AcceptUnicastHashEn	When set to '1', unicast frames that pass the imperfect hash filter are accepted.	0
4	AcceptMulticastHashEn	When set to '1', multicast frames that pass the imperfect hash filter are accepted.	0
5	AcceptPerfectEn	When set to '1', the frames with a destination address identical to the station address are accepted.	0
11:6	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
12	MagicPacketEnWoL	When set to '1', the result of the magic packet filter will generate a WoL interrupt when there is a match.	0
13	RxFilterEnWoL	When set to '1', the result of the perfect address matching filter and the imperfect hash filter will generate a WoL interrupt when there is a match.	0
31:14	-	Unused	0x0

10.13.2 Receive Filter WoL Status Register (RxFilterWoLStatus - 0x5000 0204)

The Receive Filter Wake-up on LAN Status register (RxFilterWoLStatus) is a read-only register with an address of 0x5000 0204.

[Table 167](#) lists the definition of the individual bits in the register.

Table 167. Receive Filter WoL Status register (RxFilterWoLStatus - address 0x5000 0204) bit description

Bit	Symbol	Function	Reset value
0	AcceptUnicastWoL	When the value is '1', a unicast frames caused WoL.	0
1	AcceptBroadcastWoL	When the value is '1', a broadcast frame caused WoL.	0
2	AcceptMulticastWoL	When the value is '1', a multicast frame caused WoL.	0
3	AcceptUnicastHashWoL	When the value is '1', a unicast frame that passes the imperfect hash filter caused WoL.	0
4	AcceptMulticastHashWoL	When the value is '1', a multicast frame that passes the imperfect hash filter caused WoL.	0
5	AcceptPerfectWoL	When the value is '1', the perfect address matching filter caused WoL.	0

Table 167. Receive Filter WoL Status register (RxFilterWoLStatus - address 0x5000 0204) bit description ...continued

Bit	Symbol	Function	Reset value
6	-	Unused	0x0
7	RxFilterWoL	When the value is '1', the receive filter caused WoL.	0
8	MagicPacketWoL	When the value is '1', the magic packet filter caused WoL.	0
31:9	-	Unused	0x0

The bits in this register record the cause for a WoL. Bits in RxFilterWoLStatus can be cleared by writing the RxFilterWoLClear register.

10.13.3 Receive Filter WoL Clear Register (RxFilterWoLClear - 0x5000 0208)

The Receive Filter Wake-up on LAN Clear register (RxFilterWoLClear) is a write-only register with an address of 0x5000 0208.

[Table 168](#) lists the definition of the individual bits in the register.

Table 168. Receive Filter WoL Clear register (RxFilterWoLClear - address 0x5000 0208) bit description

Bit	Symbol	Function	Reset value
0	AcceptUnicastWoLClear	When a '1' is written to one of these bits (0 to 5), the corresponding status bit in the RxFilterWoLStatus register is cleared.	0
1	AcceptBroadcastWoLClear		0
2	AcceptMulticastWoLClear		0
3	AcceptUnicastHashWoLClear		0
4	AcceptMulticastHashWoLClear		0
5	AcceptPerfectWoLClear		0
6	-	Unused	0x0
7	RxFilterWoLClear	When a '1' is written to one of these bits (7 and/or 8), the corresponding status bit in the RxFilterWoLStatus register is cleared.	0
8	MagicPacketWoLClear		0
31:9	-	Unused	0x0

The bits in this register are write-only; writing resets the corresponding bits in the RxFilterWoLStatus register.

10.13.4 Hash Filter Table LSBs Register (HashFilterL - 0x5000 0210)

The Hash Filter table LSBs register (HashFilterL) has an address of 0x5000 0210.

[Table 169](#) lists the bit definitions of the register. Details of Hash filter table use can be found in [Section 10.17.10 "Receive filtering" on page 208](#).

Table 169. Hash Filter Table LSBs register (HashFilterL - address 0x5000 0210) bit description

Bit	Symbol	Function	Reset value
31:0	HashFilterL	Bits 31:0 of the imperfect filter hash table for receive filtering.	0x0

10.13.5 Hash Filter Table MSBs Register (HashFilterH - 0x5000 0214)

The Hash Filter table MSBs register (HashFilterH) has an address of 0x5000 0214. [Table 170](#) lists the bit definitions of the register. Details of Hash filter table use can be found in [Section 10.17.10 “Receive filtering” on page 208](#).

Table 170. Hash Filter MSBs register (HashFilterH - address 0x5000 0214) bit description

Bit	Symbol	Function	Reset value
31:0	HashFilterH	Bits 63:32 of the imperfect filter hash table for receive filtering.	0x0

10.14 Module control register definitions

10.14.1 Interrupt Status Register (IntStatus - 0x5000 0FE0)

The Interrupt Status register (IntStatus) is a read-only register with an address of 0x5000 0FE0. The interrupt status register bit definition is shown in [Table 171](#). Note that all bits are flip-flops with an asynchronous set in order to be able to generate interrupts if there are wake-up events while clocks are disabled.

Table 171. Interrupt Status register (IntStatus - address 0x5000 0FE0) bit description

Bit	Symbol	Function	Reset value
0	RxOverrunInt	Interrupt set on a fatal overrun error in the receive queue. The fatal interrupt should be resolved by a Rx soft-reset. The bit is not set when there is a nonfatal overrun error.	0
1	RxErrorInt	Interrupt trigger on receive errors: AlignmentError, RangeError, LengthError, SymbolError, CRCError or NoDescriptor or Overrun.	0
2	RxFinishedInt	Interrupt triggered when all receive descriptors have been processed i.e. on the transition to the situation where ProduceIndex == ConsumeIndex.	0
3	RxDoneInt	Interrupt triggered when a receive descriptor has been processed while the Interrupt bit in the Control field of the descriptor was set.	0
4	TxUnderrunInt	Interrupt set on a fatal underrun error in the transmit queue. The fatal interrupt should be resolved by a Tx soft-reset. The bit is not set when there is a nonfatal underrun error.	0
5	TxErrorInt	Interrupt trigger on transmit errors: LateCollision, ExcessiveCollision and ExcessiveDefer, NoDescriptor or Underrun.	0
6	TxFinishedInt	Interrupt triggered when all transmit descriptors have been processed i.e. on the transition to the situation where ProduceIndex == ConsumeIndex.	0
7	TxDoneInt	Interrupt triggered when a descriptor has been transmitted while the Interrupt bit in the Control field of the descriptor was set.	0
11:8	-	Unused	0x0
12	SoftInt	Interrupt triggered by software writing a 1 to the SoftIntSet bit in the IntSet register.	0
13	WakeUpInt	Interrupt triggered by a Wake-up event detected by the receive filter.	0
31:14	-	Unused	0x0

The interrupt status register is read-only. Setting can be done via the IntSet register. Reset can be accomplished via the IntClear register.

10.14.2 Interrupt Enable Register (IntEnable - 0x5000 0FE4)

The Interrupt Enable register (IntEnable) has an address of 0x5000 0FE4. The interrupt enable register bit definition is shown in [Table 172](#).

Table 172. Interrupt Enable register (IntEnable - address 0x5000 0FE4) bit description

Bit	Symbol	Function	Reset value
0	RxOverrunIntEn	Enable for interrupt trigger on receive buffer overrun or descriptor underrun situations.	0
1	RxErrorIntEn	Enable for interrupt trigger on receive errors.	0
2	RxFinishedIntEn	Enable for interrupt triggered when all receive descriptors have been processed i.e. on the transition to the situation where ProduceIndex == ConsumeIndex.	0
3	RxDoneIntEn	Enable for interrupt triggered when a receive descriptor has been processed while the Interrupt bit in the Control field of the descriptor was set.	0
4	TxUnderrunIntEn	Enable for interrupt trigger on transmit buffer or descriptor underrun situations.	0
5	TxErrorIntEn	Enable for interrupt trigger on transmit errors.	0
6	TxFinishedIntEn	Enable for interrupt triggered when all transmit descriptors have been processed i.e. on the transition to the situation where ProduceIndex == ConsumeIndex.	0
7	TxDoneIntEn	Enable for interrupt triggered when a descriptor has been transmitted while the Interrupt bit in the Control field of the descriptor was set.	0
11:8	-	Unused	0x0
12	SoftIntEn	Enable for interrupt triggered by the SoftInt bit in the IntStatus register, caused by software writing a 1 to the SoftIntSet bit in the IntSet register.	0
13	WakeupIntEn	Enable for interrupt triggered by a Wake-up event detected by the receive filter.	0
31:14	-	Unused	0x0

10.14.3 Interrupt Clear Register (IntClear - 0x5000 0FE8)

The Interrupt Clear register (IntClear) is a write-only register with an address of 0x5000 0FE8. The interrupt clear register bit definition is shown in [Table 173](#).

Table 173. Interrupt Clear register (IntClear - address 0x5000 0FE8) bit description

Bit	Symbol	Function	Reset value
0	RxOverrunIntClr	Writing a '1' to one of these bits clears (0 to 7) the corresponding status bit in interrupt status register IntStatus.	0
1	RxErrorIntClr		0
2	RxFinishedIntClr		0
3	RxDoneIntClr		0
4	TxUnderrunIntClr		0
5	TxErrorIntClr		0
6	TxFinishedIntClr		0
7	TxDoneIntClr		0
11:8	-	Unused	0x0
12	SoftIntClr	Writing a '1' to one of these bits (12 and/or 13) clears the corresponding status bit in interrupt status register IntStatus.	0
13	WakeupIntClr		0
31:14	-	Unused	0x0

The interrupt clear register is write-only. Writing a 1 to a bit of the IntClear register clears the corresponding bit in the status register. Writing a 0 will not affect the interrupt status.

10.14.4 Interrupt Set Register (IntSet - 0x5000 0FEC)

The Interrupt Set register (IntSet) is a write-only register with an address of 0x5000 0FEC. The interrupt set register bit definition is shown in [Table 174](#).

Table 174. Interrupt Set register (IntSet - address 0x5000 0FEC) bit description

Bit	Symbol	Function	Reset value
0	RxOverrunIntSet	Writing a '1' to one of these bits (0 to 7) sets the corresponding status bit in interrupt status register IntStatus.	0
1	RxErrorIntSet		0
2	RxFinishedIntSet		0
3	RxDoneIntSet		0
4	TxUnderrunIntSet		0
5	TxErrorIntSet		0
6	TxFinishedIntSet		0
7	TxDoneIntSet		0
11:8	-	Unused	0x0
12	SoftIntSet	Writing a '1' to one of these bits (12 and/or 13) sets the corresponding status bit in interrupt status register IntStatus.	0
13	WakeupIntSet		0
31:14	-	Unused	0x0

The interrupt set register is write-only. Writing a 1 to a bit of the IntSet register sets the corresponding bit in the status register. Writing a 0 will not affect the interrupt status.

10.14.5 Power-Down Register (PowerDown - 0x5000 0FF4)

The Power-Down register (PowerDown) is used to block all AHB accesses except accesses to the Power-Down register. The register has an address of 0x5000 0FF4. The bit definition of the register is listed in [Table 175](#).

Table 175. Power-Down register (PowerDown - address 0x5000 0FF4) bit description

Bit	Symbol	Function	Reset value
30:0	-	Unused	0x0
31	PowerDownMACAHB	If true, all AHB accesses will return a read/write error, except accesses to the Power-Down register.	0

Setting the bit will return an error on all read and write accesses on the MACAHB interface except for accesses to the Power-Down register.

10.15 Descriptor and status formats

This section defines the descriptor format for the transmit and receive scatter/gather DMA engines. Each Ethernet frame can consist of one or more fragments. Each fragment corresponds to a single descriptor. The DMA managers in the Ethernet block scatter (for receive) and gather (for transmit) multiple fragments for a single Ethernet frame.

10.15.1 Receive descriptors and statuses

[Figure 20](#) depicts the layout of the receive descriptors in memory.

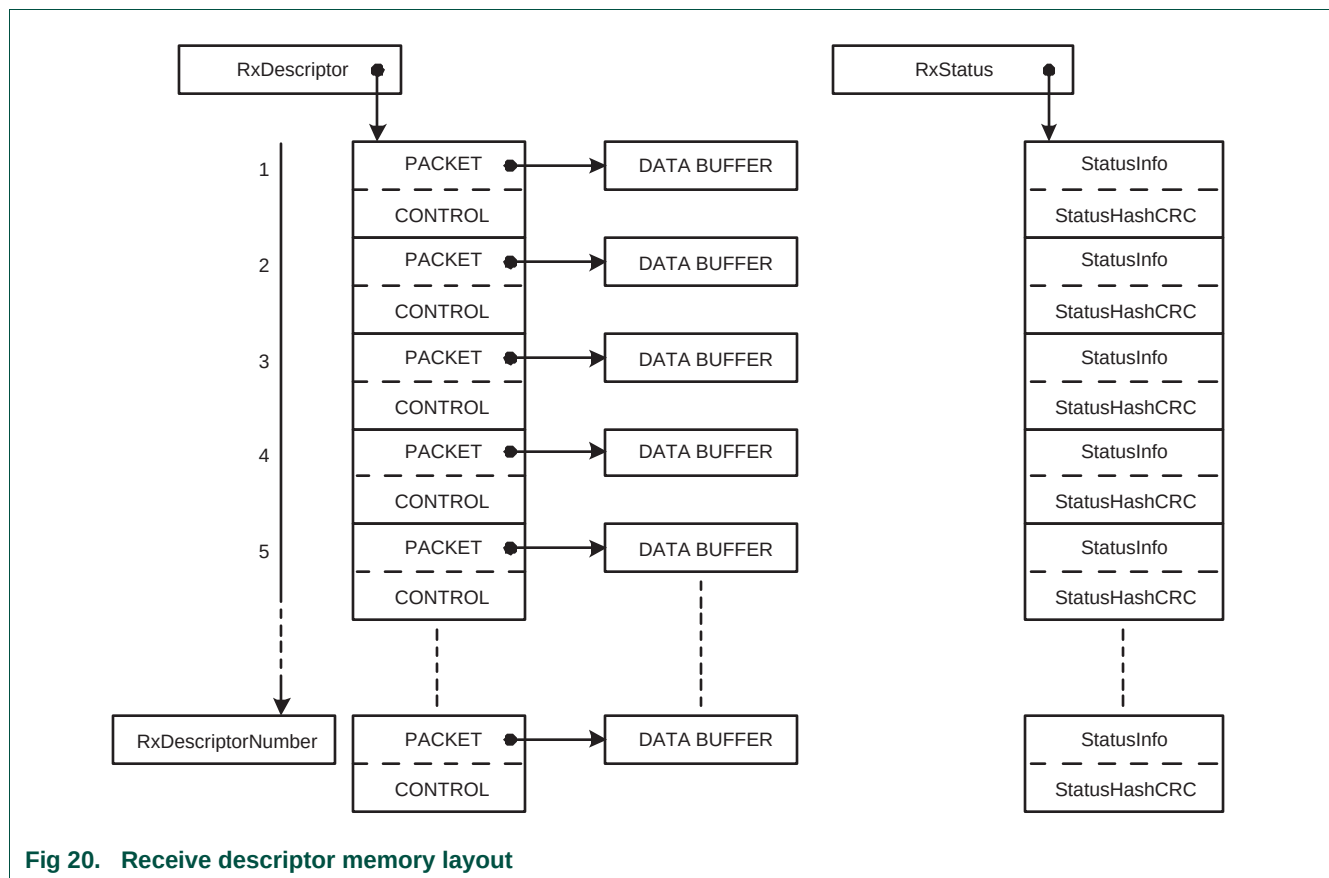


Fig 20. Receive descriptor memory layout

Receive descriptors are stored in an array in memory. The base address of the array is stored in the **RxDescriptor** register, and should be aligned on a 4 byte address boundary. The number of descriptors in the array is stored in the **RxDescriptorNumber** register using a minus one encoding style e.g. if the array has 8 elements the register value should be 7. Parallel to the descriptors there is an array of statuses. For each element of the descriptor array there is an associated status field in the status array. The base address of the status array is stored in the **RxStatus** register, and must be aligned on an 8 byte address boundary. During operation (when the receive data path is enabled) the **RxDescriptor**, **RxStatus** and **RxDescriptorNumber** registers should not be modified.

Two registers, **RxConsumeIndex** and **RxProduceIndex**, define the descriptor locations that will be used next by hardware and software. Both registers act as counters starting at 0 and wrapping when they reach the value of **RxDescriptorNumber**. The **RxProduceIndex** contains the index of the descriptor that is going to be filled with the next frame being

received. The RxConsumeIndex is programmed by software and is the index of the next descriptor that the software receive driver is going to process. When RxProduceIndex == RxConsumeIndex, the receive buffer is empty. When RxProduceIndex == RxConsumeIndex - 1 (taking wraparound into account), the receive buffer is full and newly received data would generate an overflow unless the software driver frees up one or more descriptors.

Each receive descriptor takes two word locations (8 bytes) in memory. Likewise each status field takes two words (8 bytes) in memory. Each receive descriptor consists of a pointer to the data buffer for storing receive data (Packet) and a control word (Control). The Packet field has a zero address offset, the control field has a 4 byte address offset with respect to the descriptor address as defined in [Table 176](#).

Table 176. Receive Descriptor Fields

Symbol	Address offset	Bytes	Description
Packet	0x0	4	Base address of the data buffer for storing receive data.
Control	0x4	4	Control information, see Table 177 .

The data buffer pointer (Packet) is a 32-bit, byte aligned address value containing the base address of the data buffer. The definition of the control word bits is listed in [Table 177](#).

Table 177. Receive Descriptor Control Word

Bit	Symbol	Description
10:0	Size	Size in bytes of the data buffer. This is the size of the buffer reserved by the device driver for a frame or frame fragment i.e. the byte size of the buffer pointed to by the Packet field. The size is -1 encoded e.g. if the buffer is 8 bytes the size field should be equal to 7.
30:11	-	Unused
31	Interrupt	If true generate an RxDone interrupt when the data in this frame or frame fragment and the associated status information has been committed to memory.

[Table 178](#) lists the fields in the receive status elements from the status array.

Table 178. Receive Status Fields

Symbol	Address offset	Bytes	Description
StatusInfo	0x0	4	Receive status return flags, see Table 180 .
StatusHashCRC	0x4	4	The concatenation of the destination address hash CRC and the source address hash CRC.

Each receive status consists of two words. The StatusHashCRC word contains a concatenation of the two 9-bit hash CRCs calculated from the destination and source addresses contained in the received frame. After detecting the destination and source addresses, StatusHashCRC is calculated once, then held for every fragment of the same frame.

The concatenation of the two CRCs is shown in [Table 179](#):

Table 179. Receive Status HashCRC Word

Bit	Symbol	Description
8:0	SAHashCRC	Hash CRC calculated from the source address.
15:9	-	Unused
24:16	DAHashCRC	Hash CRC calculated from the destination address.
31:25	-	Unused

The StatusInfo word contains flags returned by the MAC and flags generated by the receive data path reflecting the status of the reception. [Table 180](#) lists the bit definitions in the StatusInfo word.

Table 180. Receive status information word

Bit	Symbol	Description
10:0	RxSize	The size in bytes of the actual data transferred into one fragment buffer. In other words, this is the size of the frame or fragment as actually written by the DMA manager for one descriptor. This may be different from the Size bits of the Control field in the descriptor that indicate the size of the buffer allocated by the device driver. Size is -1 encoded e.g. if the buffer has 8 bytes the RxSize value will be 7.
17:11	-	Unused
18	ControlFrame	Indicates this is a control frame for flow control, either a pause frame or a frame with an unsupported opcode.
19	VLAN	Indicates a VLAN frame.
20	FailFilter	Indicates this frame has failed the Rx filter. These frames will not normally pass to memory. But due to the limitation of the size of the buffer, part of this frame may already be passed to memory. Once the frame is found to have failed the Rx filter, the remainder of the frame will be discarded without being passed to the memory. However, if the PassRxFilter bit in the Command register is set, the whole frame will be passed to memory.
21	Multicast	Set when a multicast frame is received.
22	Broadcast	Set when a broadcast frame is received.
23	CRCErr	The received frame had a CRC error.
24	SymbolError	The PHY reports a bit error over the PHY interface during reception.
25	LengthError	The frame length field value in the frame specifies a valid length, but does not match the actual data length.
26	RangeError ^[1]	The received packet exceeds the maximum packet size.
27	AlignmentError	An alignment error is flagged when dribble bits are detected and also a CRC error is detected. This is in accordance with IEEE std. 802.3/clause 4.3.2.
28	Overrun	Receive overrun. The adapter can not accept the data stream.
29	NoDescriptor	No new Rx descriptor is available and the frame is too long for the buffer size in the current receive descriptor.
30	LastFlag	When set to 1, indicates this descriptor is for the last fragment of a frame. If the frame consists of a single fragment, this bit is also set to 1.
31	Error	An error occurred during reception of this frame. This is a logical OR of AlignmentError, RangeError, LengthError, SymbolError, CRCErr, and Overrun.

- [1] The EMAC doesn't distinguish the frame type and frame length, so, e.g. when the IP(0x8000) or ARP(0x0806) packets are received, it compares the frame type with the max length and gives the "Range" error. In fact, this bit is not an error indication, but simply a statement by the chip regarding the status of the received frame.

For multi-fragment frames, the value of the AlignmentError, RangeError, LengthError, SymbolError and CRCError bits in all but the last fragment in the frame will be 0; likewise the value of the FailFilter, Multicast, Broadcast, VLAN and ControlFrame bits is undefined. The status of the last fragment in the frame will copy the value for these bits from the MAC. All fragment statuses will have valid LastFrag, RxSize, Error, Overrun and NoDescriptor bits.

10.15.2 Transmit descriptors and statuses

Figure 21 depicts the layout of the transmit descriptors in memory.

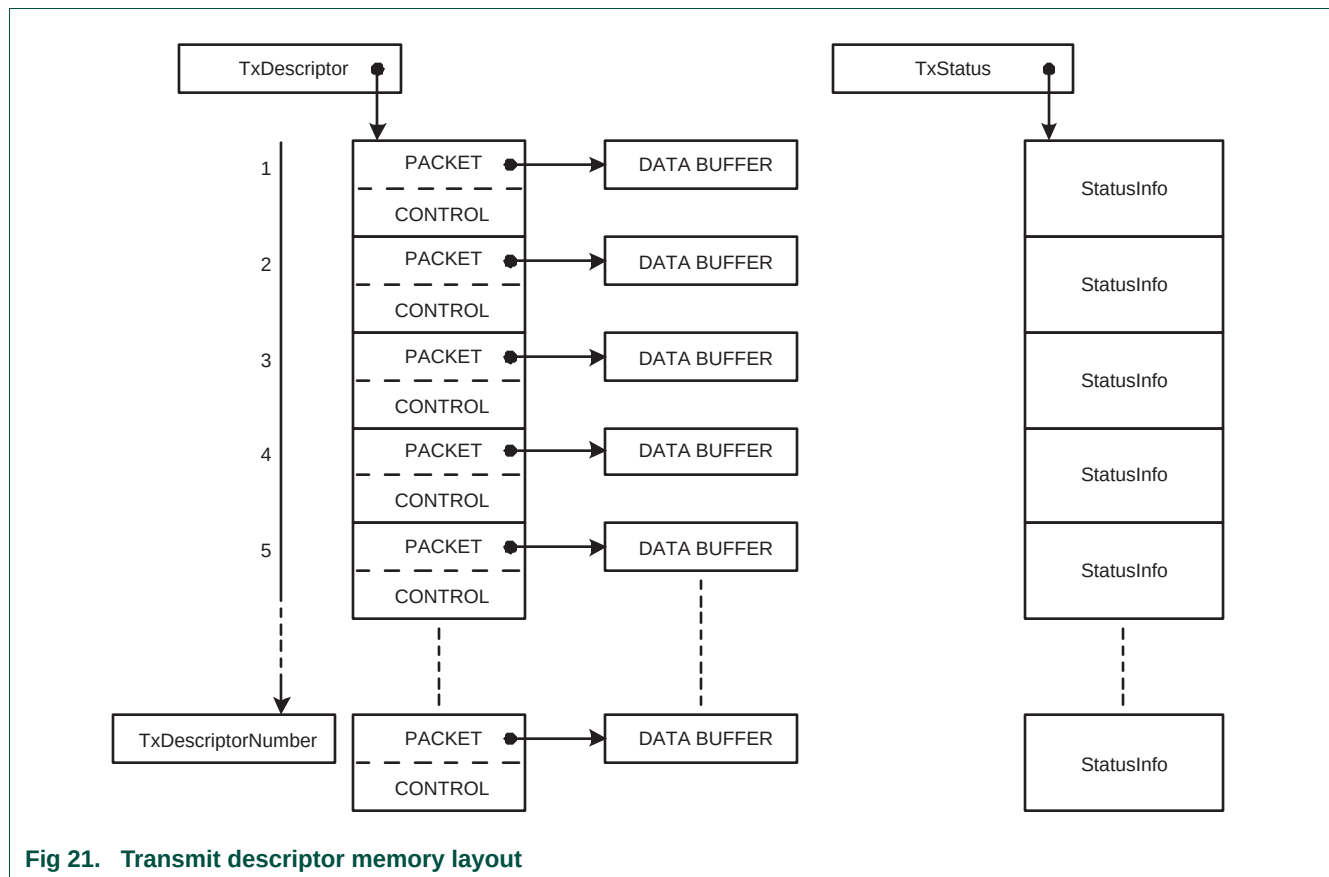


Fig 21. Transmit descriptor memory layout

Transmit descriptors are stored in an array in memory. The lowest address of the transmit descriptor array is stored in the TxDescriptor register, and must be aligned on a 4 byte address boundary. The number of descriptors in the array is stored in the TxDescriptorNumber register using a minus one encoding style i.e. if the array has 8 elements the register value should be 7. Parallel to the descriptors there is an array of statuses. For each element of the descriptor array there is an associated status field in the status array. The base address of the status array is stored in the TxStatus register, and must be aligned on a 4 byte address boundary. During operation (when the transmit data path is enabled) the TxDescriptor, TxStatus, and TxDescriptorNumber registers should not be modified.

Two registers, TxConsumeIndex and TxProduceIndex, define the descriptor locations that will be used next by hardware and software. Both register act as counters starting at 0 and wrapping when they reach the value of TxDescriptorNumber. The TxProduceIndex contains the index of the next descriptor that is going to be filled by the software driver. The TxConsumeIndex contains the index of the next descriptor going to be transmitted by the hardware. When TxProduceIndex == TxConsumeIndex, the transmit buffer is empty. When TxProduceIndex == TxConsumeIndex - 1 (taking wraparound into account), the transmit buffer is full and the software driver cannot add new descriptors until the hardware has transmitted one or more frames to free up descriptors.

Each transmit descriptor takes two word locations (8 bytes) in memory. Likewise each status field takes one word (4 bytes) in memory. Each transmit descriptor consists of a pointer to the data buffer containing transmit data (Packet) and a control word (Control). The Packet field has a zero address offset, whereas the control field has a 4 byte address offset, see [Table 181](#).

Table 181. Transmit descriptor fields

Symbol	Address offset	Bytes	Description
Packet	0x0	4	Base address of the data buffer containing transmit data.
Control	0x4	4	Control information, see Table 182 .

The data buffer pointer (Packet) is a 32-bit, byte aligned address value containing the base address of the data buffer. The definition of the control word bits is listed in [Table 182](#).

Table 182. Transmit descriptor control word

Bit	Symbol	Description
10:0	Size	Size in bytes of the data buffer. This is the size of the frame or fragment as it needs to be fetched by the DMA manager. In most cases it will be equal to the byte size of the data buffer pointed to by the Packet field of the descriptor. Size is -1 encoded e.g. a buffer of 8 bytes is encoded as the Size value 7.
25:11	-	Unused
26	Override	Per frame override. If true, bits 30:27 will override the defaults from the MAC internal registers. If false, bits 30:27 will be ignored and the default values from the MAC will be used.
27	Huge	If true, enables huge frame, allowing unlimited frame sizes. When false, prevents transmission of more than the maximum frame length (MAXF[15:0]).
28	Pad	If true, pad short frames to 64 bytes.
29	CRC	If true, append a hardware CRC to the frame.
30	Last	If true, indicates that this is the descriptor for the last fragment in the transmit frame. If false, the fragment from the next descriptor should be appended.
31	Interrupt	If true, a TxDone interrupt will be generated when the data in this frame or frame fragment has been sent and the associated status information has been committed to memory.

[Table 183](#) shows the one field transmit status.

Table 183. Transmit status fields

Symbol	Address offset	Bytes	Description
StatusInfo	0x0	4	Transmit status return flags, see Table 184 .

The transmit status consists of one word which is the StatusInfo word. It contains flags returned by the MAC and flags generated by the transmit data path reflecting the status of the transmission. [Table 184](#) lists the bit definitions in the StatusInfo word.

Table 184. Transmit status information word

Bit	Symbol	Description
20:0	-	Unused
24:21	CollisionCount	The number of collisions this packet incurred, up to the Retransmission Maximum.
25	Defer	This packet incurred deferral, because the medium was occupied. This is not an error unless excessive deferral occurs.
26	ExcessiveDefer	This packet incurred deferral beyond the maximum deferral limit and was aborted.
27	ExcessiveCollision	Indicates this packet exceeded the maximum collision limit and was aborted.
28	LateCollision	An Out of window Collision was seen, causing packet abort.
29	Underrun	A Tx underrun occurred due to the adapter not producing transmit data.
30	NoDescriptor	The transmit stream was interrupted because a descriptor was not available.
31	Error	An error occurred during transmission. This is a logical OR of Underrun, LateCollision, ExcessiveCollision, and ExcessiveDefer.

For multi-fragment frames, the value of the LateCollision, ExcessiveCollision, ExcessiveDefer, Defer and CollisionCount bits in all but the last fragment in the frame will be 0. The status of the last fragment in the frame will copy the value for these bits from the MAC. All fragment statuses will have valid Error, NoDescriptor and Underrun bits.

10.16 Ethernet block functional description

This section defines the functions of the DMA capable 10/100 Ethernet MAC. After introducing the DMA concepts of the Ethernet block, and a description of the basic transmit and receive functions, this section elaborates on advanced features such as flow control, receive filtering, etc.

10.16.1 Overview

The Ethernet block can transmit and receive Ethernet packets from an off-chip Ethernet PHY connected through the RMII interface.

Typically during system start-up, the Ethernet block will be initialized. Software initialization of the Ethernet block should include initialization of the descriptor and status arrays as well as the receiver fragment buffers.

Remark: When initializing the Ethernet block, it is important to first configure the PHY and ensure that reference clocks (ENET_REF_CLK signal in RMII mode, or both ENET_RX_CLK and ENET_TX_CLK signals in MII mode) are present at the external pins and connected to the EMAC module (selecting the appropriate pins using the PINSEL registers) prior to continuing with Ethernet configuration. Otherwise the CPU can become locked and no further functionality will be possible. This will cause JTAG lose communication with the target, if debug mode is being used.

To transmit a packet the software driver has to set up the appropriate Control registers and a descriptor to point to the packet data buffer before transferring the packet to hardware by incrementing the TxProduceIndex register. After transmission, hardware will increment TxConsumeIndex and optionally generate an interrupt.

The hardware will receive packets from the PHY and apply filtering as configured by the software driver. While receiving a packet the hardware will read a descriptor from memory to find the location of the associated receiver data buffer. Receive data is written in the data buffer and receive status is returned in the receive descriptor status word. Optionally an interrupt can be generated to notify software that a packet has been received. Note that the DMA manager will prefetch and buffer up to three descriptors.

10.16.2 AHB interface

The registers of the Ethernet block connect to an AHB slave interface to allow access to the registers from the CPU.

The AHB interface has a 32-bit data path, which supports only word accesses and has an address aperture of 4 kB. [Table 129](#) lists the registers of the Ethernet block.

All AHB write accesses to registers are posted except for accesses to the IntSet, IntClear and IntEnable registers. AHB write operations are executed in order.

If the PowerDown bit of the PowerDown register is set, all AHB read and write accesses will return a read or write error except for accesses to the PowerDown register.

Bus Errors

The Ethernet block generates errors for several conditions:

- The AHB interface will return a read error when there is an AHB read access to a write-only register; likewise a write error is returned when there is an AHB write access to the read-only register. An AHB read or write error will be returned on AHB read or write accesses to reserved registers. These errors are propagated back to the CPU. Registers defined as read-only and write-only are identified in [Table 129](#).
- If the PowerDown bit is set all accesses to AHB registers will result in an error response except for accesses to the PowerDown register.

10.17 Interrupts

The Ethernet block has a single interrupt request output to the CPU (via the NVIC).

The interrupt service routine must read the IntStatus register to determine the origin of the interrupt. All interrupt statuses can be set by software writing to the IntSet register; statuses can be cleared by software writing to the IntClear register.

The transmit and receive data paths can only set interrupt statuses, they cannot clear statuses. The SoftInt interrupt cannot be set by hardware and can be used by software for test purposes.

10.17.1 Direct Memory Access (DMA)

Descriptor arrays

The Ethernet block includes two DMA managers. The DMA managers make it possible to transfer frames directly to and from memory with little support from the processor and without the need to trigger an interrupt for each frame.

The DMA managers work with arrays of frame descriptors and statuses that are stored in memory. The descriptors and statuses act as an interface between the Ethernet hardware and the device driver software. There is one descriptor array for receive frames and one descriptor array for transmit frames. Using buffering for frame descriptors, the memory traffic and memory bandwidth utilization of descriptors can be kept small.

Each frame descriptor contains two 32-bit fields: the first field is a pointer to a data buffer containing a frame or a fragment, whereas the second field is a control word related to that frame or fragment.

The software driver must write the base addresses of the descriptor and status arrays in the TxDescriptor/RxDescriptor and TxStatus/RxStatus registers. The number of descriptors/statuses in each array must be written in the TxDescriptorNumber/RxDescriptorNumber registers. The number of descriptors in an array corresponds to the number of statuses in the associated status array.

Transmit descriptor arrays, receive descriptor arrays and transmit status arrays must be aligned on a 4 byte (32bit) address boundary, while the receive status array must be aligned on a 8 byte (64bit) address boundary.

Ownership of descriptors

Both device driver software and Ethernet hardware can read and write the descriptor arrays at the same time in order to produce and consume descriptors. A descriptor is "owned" either by the device driver or by the Ethernet hardware. Only the owner of a descriptor reads or writes its value. Typically, the sequence of use and ownership of descriptors and statuses is as follows: a descriptor is owned and set up by the device driver; ownership of the descriptor/status is passed by the device driver to the Ethernet block, which reads the descriptor and writes information to the status field; the Ethernet block passes ownership of the descriptor back to the device driver, which uses the status information and then recycles the descriptor to be used for another frame. Software must pre-allocate the memory used to hold the descriptor arrays.

Software can hand over ownership of descriptors and statuses to the hardware by incrementing (and wrapping if on the array boundary) the TxProduceIndex/RxConsumeIndex registers. Hardware hands over descriptors and status to software by updating the TxConsumeIndex/ RxProduceIndex registers.

After handing over a descriptor to the receive and transmit DMA hardware, device driver software should not modify the descriptor or reclaim the descriptor by decrementing the TxProduceIndex/ RxConsumeIndex registers because descriptors may have been prefetched by the hardware. In this case the device driver software will have to wait until the frame has been transmitted or the device driver has to soft-reset the transmit and/or receive data paths which will also reset the descriptor arrays.

Sequential order with wrap-around

When descriptors are read from and statuses are written to the arrays, this is done in sequential order with wrap-around. Sequential order means that when the Ethernet block has finished reading/writing a descriptor/status, the next descriptor/status it reads/writes is

the one at the next higher, adjacent memory address. Wrap around means that when the Ethernet block has finished reading/writing the last descriptor/status of the array (with the highest memory address), the next descriptor/status it reads/writes is the first descriptor/status of the array at the base address of the array.

Full and Empty state of descriptor arrays

The descriptor arrays can be empty, partially full or full. A descriptor array is empty when all descriptors are owned by the producer. A descriptor array is partially full if both producer and consumer own part of the descriptors and both are busy processing those descriptors. A descriptor array is full when all descriptors (except one) are owned by the consumer, so that the producer has no more room to process frames. Ownership of descriptors is indicated with the use of a consume index and a produce index. The produce index is the first element of the array owned by the producer. It is also the index of the array element that is next going to be used by the producer of frames (it may already be busy using it and subsequent elements). The consume index is the first element of the array that is owned by the consumer. It is also the number of the array element next to be consumed by the consumer of frames (it and subsequent elements may already be in the process of being consumed). If the consume index and the produce index are equal, the descriptor array is empty and all array elements are owned by the producer. If the consume index equals the produce index plus one, then the array is full and all array elements (except the one at the produce index) are owned by the consumer. With a full descriptor array, still one array element is kept empty, to be able to easily distinguish the full or empty state by looking at the value of the produce index and consume index. An array must have at least 2 elements to be able to indicate a full descriptor array with a produce index of value 0 and a consume index of value 1. The wrap around of the arrays is taken into account when determining if a descriptor array is full, so a produce index that indicates the last element in the array and a consume index that indicates the first element in the array, also means the descriptor array is full. When the produce index and the consume index are unequal and the consume index is not the produce index plus one (with wrap around taken into account), then the descriptor array is partially full and both the consumer and producer own enough descriptors to be able to operate actively on the descriptor array.

Interrupt bit

The descriptors have an Interrupt bit, which is programmed by software. When the Ethernet block is processing a descriptor and finds this bit set, it will allow triggering an interrupt (after committing status to memory) by passing the RxDoneInt or TxDoneInt bits in the IntStatus register to the interrupt output pin. If the Interrupt bit is not set in the descriptor, then the RxDoneInt or TxDoneInt are not set and no interrupt is triggered (note that the corresponding bits in IntEnable must also be set to trigger interrupts). This offers flexible ways of managing the descriptor arrays. For instance, the device driver could add 10 frames to the Tx descriptor array, and set the Interrupt bit in descriptor number 5 in the descriptor array. This would invoke the interrupt service routine before the transmit descriptor array is completely exhausted. The device driver could add another batch of frames to the descriptor array, without interrupting continuous transmission of frames.

Frame fragments

For maximum flexibility in frame storage, frames can be split up into multiple frame fragments with fragments located in different places in memory. In this case one descriptor is used for each frame fragment. So, a descriptor can point to a single frame or

to a fragment of a frame. By using fragments, scatter/gather DMA can be done: transmit frames are gathered from multiple fragments in memory and receive frames can be scattered to multiple fragments in memory.

By stringing together fragments it is possible to create large frames from small memory areas. Another use of fragments is to be able to locate a frame header and frame payload in different places and to concatenate them without copy operations in the device driver.

For transmissions, the Last bit in the descriptor Control field indicates if the fragment is the last in a frame; for receive frames, the LastFrag bit in the StatusInfo field of the status words indicates if the fragment is the last in the frame. If the Last(Frag) bit is 0 the next descriptor belongs to the same Ethernet frame, If the Last(Frag) bit is 1 the next descriptor is a new Ethernet frame.

10.17.2 Initialization

After reset, the Ethernet software driver needs to initialize the Ethernet block. During initialization the software needs to:

- Remove the soft reset condition from the MAC
- Configure the PHY via the MIIM interface of the MAC.

Remark: it is important to configure the PHY and insure that reference clocks (ENET_REF_CLK signal in RMII mode, or both ENET_RX_CLK and ENET_TX_CLK signals in MII mode) are present at the external pins and connected to the EMAC module (selecting the appropriate pins using the PINSEL registers) prior to continuing with Ethernet configuration. Otherwise the CPU can become locked and no further functionality will be possible. This will cause JTAG lose communication with the target, if debug mode is being used.

- Select RMII mode
- Configure the transmit and receive DMA engines, including the descriptor arrays
- Configure the host registers (MAC1,MAC2 etc.) in the MAC
- Enable the receive and transmit data paths

Depending on the PHY, the software needs to initialize registers in the PHY via the MII Management interface. The software can read and write PHY registers by programming the MCFG, MCMD, MADR registers of the MAC. Write data should be written to the MWTD register; read data and status information can be read from the MRDD and MIND registers.

The Ethernet block supports RMII PHYs. During initialization software must select RMII mode by programming the Command register.

Before switching to RMII mode the default soft reset (MAC1 register bit 15) has to be de-asserted. The phy_ref_clk must be running and internally connected during this operation.

Transmit and receive DMA engines should be initialized by the device driver by allocating the descriptor and status arrays in memory. Transmit and receive functions have their own dedicated descriptor and status arrays. The base addresses of these arrays need to be programmed in the TxDescriptor/TxStatus and RxDescriptor/RxStatus registers. The number of descriptors in an array matches the number of statuses in an array.

Please note that the transmit descriptors, receive descriptors and receive statuses are 8 bytes each while the transmit statuses are 4 bytes each. All descriptor arrays and transmit statuses need to be aligned on 4 byte boundaries; receive status arrays need to be aligned on 8 byte boundaries. The number of descriptors in the descriptor arrays needs to be written to the TxDescriptorNumber/RxDescriptorNumber registers using a -1 encoding i.e. the value in the registers is the number of descriptors minus one e.g. if the descriptor array has 4 descriptors the value of the number of descriptors register should be 3.

After setting up the descriptor arrays, frame buffers need to be allocated for the receive descriptors before enabling the receive data path. The Packet field of the receive descriptors needs to be filled with the base address of the frame buffer of that descriptor. Amongst others the Control field in the receive descriptor needs to contain the size of the data buffer using -1 encoding.

The receive data path has a configurable filtering function for discarding/ignoring specific Ethernet frames. The filtering function should also be configured during initialization.

After an assertion of the hardware reset, the soft reset bit in the MAC will be asserted. The soft reset condition must be removed before the Ethernet block can be enabled.

Enabling of the receive function is located in two places. The receive DMA manager needs to be enabled and the receive data path of the MAC needs to be enabled. To prevent overflow in the receive DMA engine the receive DMA engine should be enabled by setting the RxEnable bit in the Command register before enabling the receive data path in the MAC by setting the RECEIVE ENABLE bit in the MAC1 register.

The transmit DMA engine can be enabled at any time by setting the TxEnable bit in the Command register.

Before enabling the data paths, several options can be programmed in the MAC, such as automatic flow control, transmit to receive loop-back for verification, full/half duplex modes, etc.

Base addresses of descriptor arrays and descriptor array sizes cannot be modified without a (soft) reset of the receive and transmit data paths.

10.17.3 Transmit process

Overview

This section outlines the transmission process.

Device driver sets up descriptors and data

If the descriptor array is full the device driver should wait for the descriptor arrays to become not full before writing to a descriptor in the descriptor array. If the descriptor array is not full, the device driver should use the descriptor numbered TxProduceIndex of the array pointed to by TxDescriptor.

The Packet pointer in the descriptor is set to point to a data frame or frame fragment to be transmitted. The Size field in the Command field of the descriptor should be set to the number of bytes in the fragment buffer, -1 encoded. Additional control information can be indicated in the Control field in the descriptor (bits Interrupt, Last, CRC, Pad).

After writing the descriptor the descriptor needs to be handed over to the hardware by incrementing (and possibly wrapping) the TxProduceIndex register.

If the transmit data path is disabled, the device driver should not forget to enable the transmit data path by setting the TxEnable bit in the Command register.

When there is a multi-fragment transmission for fragments other than the last, the Last bit in the descriptor must be set to 0; for the last fragment the Last bit must be set to 1. To trigger an interrupt when the frame has been transmitted and transmission status has been committed to memory, set the Interrupt bit in the descriptor Control field to 1. To have the hardware add a CRC in the frame sequence control field of this Ethernet frame, set the CRC bit in the descriptor. This should be done if the CRC has not already been added by software. To enable automatic padding of small frames to the minimum required frame size, set the Pad bit in the Control field of the descriptor to 1. In typical applications bits CRC and Pad are both set to 1.

The device driver can set up interrupts using the IntEnable register to wait for a signal of completion from the hardware or can periodically inspect (poll) the progress of transmission. It can also add new frames at the end of the descriptor array, while hardware consumes descriptors at the start of the array.

The device driver can stop the transmit process by resetting the TxEnable bit in the Command register to 0. The transmission will not stop immediately; frames already being transmitted will be transmitted completely and the status will be committed to memory before deactivating the data path. The status of the transmit data path can be monitored by the device driver reading the TxStatus bit in the Status register.

As soon as the transmit data path is enabled and the corresponding TxConsumeIndex and TxProduceIndex are not equal i.e. the hardware still needs to process frames from the descriptor array, the TxStatus bit in the Status register will return to 1 (active).

Tx DMA manager reads the Tx descriptor array

When the TxEnable bit is set, the Tx DMA manager reads the descriptors from memory at the address determined by TxDescriptor and TxConsumeIndex. The number of descriptors requested is determined by the total number of descriptors owned by the hardware: TxProduceIndex - TxConsumeIndex. Block transferring descriptors minimizes memory loading. Read data returned from memory is buffered and consumed as needed.

Tx DMA manager transmits data

After reading the descriptor the transmit DMA engine reads the associated frame data from memory and transmits the frame. After transfer completion, the Tx DMA manager writes status information back to the StatusInfo and StatusHashCRC words of the status field. The value of the TxConsumeIndex is only updated after status information has been committed to memory, which is checked by an internal tag protocol in the memory interface. The Tx DMA manager continues to transmit frames until the descriptor array is empty. If the transmit descriptor array is empty the TxStatus bit in the Status register will return to 0 (inactive). If the descriptor array is empty the Ethernet hardware will set the TxFinishedInt bit of the IntStatus register. The transmit data path will still be enabled.

The Tx DMA manager inspects the Last bit of the descriptor Control field when loading the descriptor. If the Last bit is 0, this indicates that the frame consists of multiple fragments. The Tx DMA manager gathers all the fragments from the host memory, visiting a string of

frame descriptors, and sends them out as one Ethernet frame on the Ethernet connection. When the Tx DMA manager finds a descriptor with the Last bit in the Control field set to 1, this indicates the last fragment of the frame and thus the end of the frame is found.

Update ConsumeIndex

Each time the Tx DMA manager commits a status word to memory it completes the transmission of a descriptor and it increments the TxConsumeIndex (taking wrap around into account) to hand the descriptor back to the device driver software. Software can re-use the descriptor for new transmissions after hardware has handed it back.

The device driver software can keep track of the progress of the DMA manager by reading the TxConsumeIndex register to see how far along the transmit process is. When the Tx descriptor array is emptied completely, the TxConsumeIndex register retains its last value.

Write transmission status

After the frame has been transmitted over the RMII bus, the StatusInfo word of the frame descriptor is updated by the DMA manager.

If the descriptor is for the last fragment of a frame (or for the whole frame if there are no fragments), then depending on the success or failure of the frame transmission, error flags (Error, LateCollision, ExcessiveCollision, Underrun, ExcessiveDefer, Defer) are set in the status. The CollisionCount field is set to the number of collisions the frame incurred, up to the Retransmission Maximum programmed in the Collision window/retry register of the MAC.

Statuses for all but the last fragment in the frame will be written as soon as the data in the frame has been accepted by the Tx DMA manager. Even if the descriptor is for a frame fragment other than the last fragment, the error flags are returned via the AHB interface. If the Ethernet block detects a transmission error during transmission of a (multi-fragment) frame, all remaining fragments of the frame are still read via the AHB interface. After an error, the remaining transmit data is discarded by the Ethernet block. If there are errors during transmission of a multi-fragment frame the error statuses will be repeated until the last fragment of the frame. Statuses for all but the last fragment in the frame will be written as soon as the data in the frame has been accepted by the Tx DMA manager. These may include error information if the error is detected early enough. The status for the last fragment in the frame will only be written after the transmission has completed on the Ethernet connection. Thus, the status for the last fragment will always reflect any error that occurred anywhere in the frame.

The status of the last frame transmission can also be inspected by reading the TSV0 and TSV1 registers. These registers do not report statuses on a fragment basis and do not store information of previously sent frames. They are provided primarily for debug purposes, because the communication between driver software and the Ethernet block takes place through the frame descriptors. The status registers are valid as long as the internal status of the MAC is valid and should typically only be read when the transmit and receive processes are halted.

Transmission error handling

If an error occurs during the transmit process, the Tx DMA manager will report the error via the transmission StatusInfo word written in the Status array and the IntStatus interrupt status register.

The transmission can generate several types of errors: LateCollision, ExcessiveCollision, ExcessiveDefer, Underrun, and NoDescriptor. All have corresponding bits in the transmission StatusInfo word. In addition to the separate bits in the StatusInfo word, LateCollision, ExcessiveCollision, and ExcessiveDefer are ORed together into the Error bit of the Status. Errors are also propagated to the IntStatus register; the TxError bit in the IntStatus register is set in the case of a LateCollision, ExcessiveCollision, ExcessiveDefer, or NoDescriptor error; Underrun errors are reported in the TxUnderrun bit of the IntStatus register.

Underrun errors can have three causes:

- The next fragment in a multi-fragment transmission is not available. This is a nonfatal error. A NoDescriptor status will be returned on the previous fragment and the TxError bit in IntStatus will be set.
- The transmission fragment data is not available when the Ethernet block has already started sending the frame. This is a nonfatal error. An Underrun status will be returned on transfer and the TxError bit in IntStatus will be set.
- The flow of transmission statuses stalls and a new status has to be written while a previous status still waits to be transferred across the memory interface. This is a fatal error which can only be resolved by a soft reset of the hardware.

The first and second situations are nonfatal and the device driver has to re-send the frame or have upper software layers re-send the frame. In the third case the hardware is in an undefined state and needs to be soft reset by setting the TxReset bit in the Command register.

After reporting a LateCollision, ExcessiveCollision, ExcessiveDefer or Underrun error, the transmission of the erroneous frame will be aborted, remaining transmission data and frame fragments will be discarded and transmission will continue with the next frame in the descriptor array.

Device drivers should catch the transmission errors and take action.

Transmit triggers interrupts

The transmit data path can generate four different interrupt types:

- If the Interrupt bit in the descriptor Control field is set, the Tx DMA will set the TxDoneInt bit in the IntStatus register after sending the fragment and committing the associated transmission status to memory. Even if a descriptor (fragment) is not the last in a multi-fragment frame the Interrupt bit in the descriptor can be used to generate an interrupt.
- If the descriptor array is empty while the Ethernet hardware is enabled the hardware will set the TxFinishedInt bit of the IntStatus register.
- If the AHB interface does not consume the transmission statuses at a sufficiently high bandwidth the transmission may underrun in which case the TxUnderrun bit will be set in the IntStatus register. This is a fatal error which requires a soft reset of the transmission queue.

- In the case of a transmission error (LateCollision, ExcessiveCollision, or ExcessiveDefer) or a multi-fragment frame where the device driver did provide the initial fragments but did not provide the rest of the fragments (NoDescriptor) or in the case of a nonfatal overrun, the hardware will set the TxErrorInt bit of the IntStatus register.

All of the above interrupts can be enabled and disabled by setting or resetting the corresponding bits in the IntEnable register. Enabling or disabling does not affect the IntStatus register contents, only the propagation of the interrupt status to the CPU (via the NVIC).

The interrupts, either of individual frames or of the whole list, are a good means of communication between the DMA manager and the device driver, triggering the device driver to inspect the status words of descriptors that have been processed.

Transmit example

[Figure 22](#) illustrates the transmit process in an example transmitting uses a frame header of 8 bytes and a frame payload of 12 bytes.

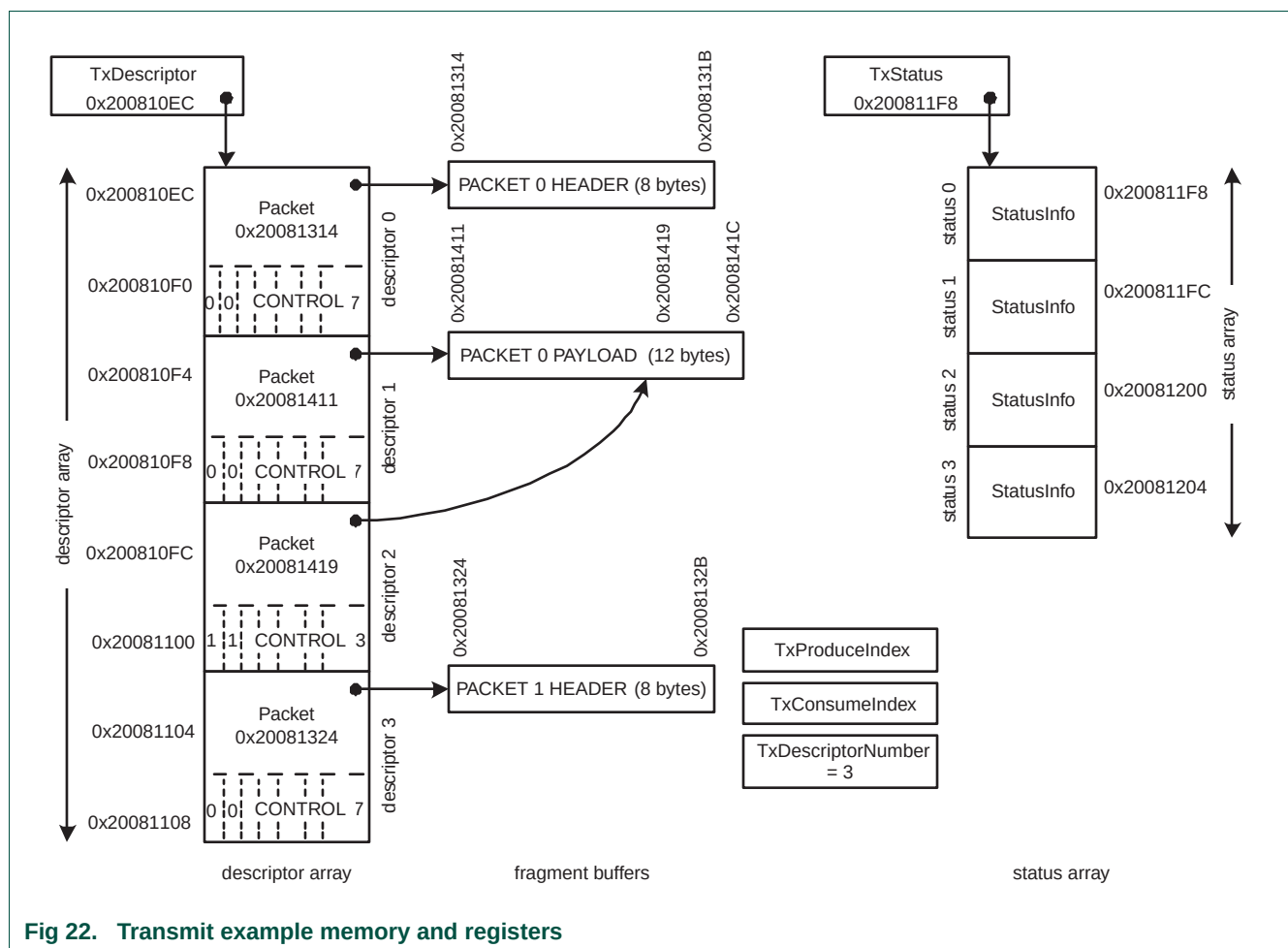


Fig 22. Transmit example memory and registers

After reset the values of the DMA registers will be zero. During initialization the device driver will allocate the descriptor and status array in memory. In this example, an array of four descriptors is allocated; the array is 4x2x4 bytes and aligned on a 4 byte address

boundary. Since the number of descriptors matches the number of statuses the status array consists of four elements; the array is 4x1x4 bytes and aligned on a 4 byte address boundary. The device driver writes the base address of the descriptor array (0x2008 10EC) to the TxDescriptor register and the base address of the status array (0x2008 11F8) to the TxStatus register. The device driver writes the number of descriptors and statuses minus 1(3) to the TxDescriptorNumber register. The descriptors and statuses in the arrays need not be initialized, yet.

At this point, the transmit data path may be enabled by setting the TxEnable bit in the Command register. If the transmit data path is enabled while there are no further frames to send the TxFinishedInt interrupt flag will be set. To reduce the processor interrupt load only the desired interrupts can be enabled by setting the relevant bits in the IntEnable register.

Now suppose application software wants to transmit a frame of 12 bytes using a TCP/IP protocol (in real applications frames will be larger than 12 bytes). The TCP/IP stack will add a header to the frame. The frame header need not be immediately in front of the payload data in memory. The device driver can program the Tx DMA to collect header and payload data. To do so, the device driver will program the first descriptor to point at the frame header; the Last flag in the descriptor will be set to false/0 to indicate a multi-fragment transmission. The device driver will program the next descriptor to point at the actual payload data. The maximum size of a payload buffer is 2 kB so a single descriptor suffices to describe the payload buffer. For the sake of the example though the payload is distributed across two descriptors. After the first descriptor in the array describing the header, the second descriptor in the array describes the initial 8 bytes of the payload; the third descriptor in the array describes the remaining 4 bytes of the frame. In the third descriptor the Last bit in the Control word is set to true/1 to indicate it is the last descriptor in the frame. In this example the Interrupt bit in the descriptor Control field is set in the last fragment of the frame in order to trigger an interrupt after the transmission completed. The Size field in the descriptor's Control word is set to the number of bytes in the fragment buffer, -1 encoded.

Note that in real device drivers, the payload will typically only be split across multiple descriptors if it is more than 2 kB. Also note that transmission payload data is forwarded to the hardware without the device driver copying it (zero copy device driver).

After setting up the descriptors for the transaction the device driver increments the TxProduceIndex register by 3 since three descriptors have been programmed. If the transmit data path was not enabled during initialization the device driver needs to enable the data path now.

If the transmit data path is enabled the Ethernet block will start transmitting the frame as soon as it detects the TxProduceIndex is not equal to TxConsumeIndex - both were zero after reset. The Tx DMA will start reading the descriptors from memory. The memory system will return the descriptors and the Ethernet block will accept them one by one while reading the transmit data fragments.

As soon as transmission read data is returned from memory, the Ethernet block will try to start transmission on the Ethernet connection via the RMII interface.

After transmitting each fragment of the frame the Tx DMA will write the status of the fragment's transmission. Statuses for all but the last fragment in the frame will be written as soon as the data in the frame has been accepted by the Tx DMA manager. The status for the last fragment in the frame will only be written after the transmission has completed on the Ethernet connection.

Since the Interrupt bit in the descriptor of the last fragment is set, after committing the status of the last fragment to memory the Ethernet block will trigger a TxDoneInt interrupt, which triggers the device driver to inspect the status information.

In this example the device driver cannot add new descriptors as long as the Ethernet block has not incremented the TxConsumeIndex because the descriptor array is full (even though one descriptor is not programmed yet). Only after the hardware commits the status for the first fragment to memory and the TxConsumeIndex is set to 1 by the DMA manager can the device driver program the next (the fourth) descriptor. The fourth descriptor can already be programmed before completely transmitting the first frame.

In this example the hardware adds the CRC to the frame. If the device driver software adds the CRC, the CRC trailer can be considered another frame fragment which can be added by doing another gather DMA.

Each data byte is transmitted across the RMII interface as four 2-bit values. The Ethernet block adds the preamble, frame delimiter leader, and the CRC trailer if hardware CRC is enabled. Once transmission on the RMII interface commences the transmission cannot be interrupted without generating an underrun error, which is why descriptors and data read commands are issued as soon as possible and pipelined.

Using an RMII PHY, the data communication between the Ethernet block and the PHY is communicated at 50 MHz. In 10 Mbps mode data will only be transmitted once every 10 clock cycles.

10.17.4 Receive process

This section outlines the receive process including the activities in the device driver software.

Device driver sets up descriptors

After initializing the receive descriptor and status arrays to receive frames from the Ethernet connection, the receive data path should be enabled in the MAC1 register and the Control register.

During initialization, each Packet pointer in the descriptors is set to point to a data fragment buffer. The size of the buffer is stored in the Size bits of the Control field of the descriptor. Additionally, the Control field in the descriptor has an Interrupt bit. The Interrupt bit allows generation of an interrupt after a fragment buffer has been filled and its status has been committed to memory.

After the initialization and enabling of the receive data path, all descriptors are owned by the receive hardware and should not be modified by the software unless hardware hands over the descriptor by incrementing the RxProduceIndex, indicating that a frame has been received. The device driver is allowed to modify the descriptors after a (soft) reset of the receive data path.

Rx DMA manager reads Rx descriptor arrays

When the RxEnable bit in the Command register is set, the Rx DMA manager reads the descriptors from memory at the address determined by RxDescriptor and RxProduceIndex. The Ethernet block will start reading descriptors even before actual receive data arrives on the RMII interface (descriptor prefetching). The block size of the descriptors to be read is determined by the total number of descriptors owned by the hardware: RxConsumeIndex - RxProduceIndex - 1. Block transferring of descriptors minimizes memory load. Read data returned from memory is buffered and consumed as needed.

RX DMA manager receives data

After reading the descriptor, the receive DMA engine waits for the MAC to return receive data from the RMII interface that passes the receive filter. Receive frames that do not match the filtering criteria are not passed to memory. Once a frame passes the receive filter, the data is written in the fragment buffer associated with the descriptor. The Rx DMA does not write beyond the size of the buffer. When a frame is received that is larger than a descriptor's fragment buffer, the frame will be written to multiple fragment buffers of consecutive descriptors. In the case of a multi-fragment reception, all but the last fragment in the frame will return a status where the LastFrag bit is set to 0. Only on the last fragment of a frame the LastFrag bit in the status will be set to 1. If a fragment buffer is the last of a frame, the buffer may not be filled completely. The first receive data of the next frame will be written to the fragment buffer of the next descriptor.

After receiving a fragment, the Rx DMA manager writes status information back to the StatusInfo and StatusHashCRC words of the status. The Ethernet block writes the size in bytes of a descriptor's fragment buffer in the RxSize field of the Status word. The value of the RxProduceIndex is only updated after the fragment data and the fragment status information has been committed to memory, which is checked by an internal tag protocol in the memory interface. The Rx DMA manager continues to receive frames until the descriptor array is full. If the descriptor array is full, the Ethernet hardware will set the RxFinishedInt bit of the IntStatus register. The receive data path will still be enabled. If the receive descriptor array is full any new receive data will generate an overflow error and interrupt.

Update ProduceIndex

Each time the Rx DMA manager commits a data fragment and the associated status word to memory, it completes the reception of a descriptor and increments the RxProduceIndex (taking wrap around into account) in order to hand the descriptor back to the device driver software. Software can re-use the descriptor for new receptions by handing it back to hardware when the receive data has been processed.

The device driver software can keep track of the progress of the DMA manager by reading the RxProduceIndex register to see how far along the receive process is. When the Rx descriptor array is emptied completely, the RxProduceIndex retains its last value.

Write reception status

After the frame has been received from the RMII bus, the StatusInfo and StatusHashCRC words of the frame descriptor are updated by the DMA manager.

If the descriptor is for the last fragment of a frame (or for the whole frame if there are no fragments), then depending on the success or failure of the frame reception, error flags (Error, NoDescriptor, Overrun, AlignmentError, RangeError, LengthError, SymbolError, or CRCError) are set in StatusInfo. The RxSize field is set to the number of bytes actually written to the fragment buffer, -1 encoded. For fragments not being the last in the frame the RxSize will match the size of the buffer. The hash CRCs of the destination and source addresses of a packet are calculated once for all the fragments belonging to the same packet and then stored in every StatusHashCRC word of the statuses associated with the corresponding fragments. If the reception reports an error, any remaining data in the receive frame is discarded and the LastFrag bit will be set in the receive status field, so the error flags in all but the last fragment of a frame will always be 0.

The status of the last received frame can also be inspected by reading the RSV register. The register does not report statuses on a fragment basis and does not store information of previously received frames. RSV is provided primarily for debug purposes, because the communication between driver software and the Ethernet block takes place through the frame descriptors.

Reception error handling

When an error occurs during the receive process, the Rx DMA manager will report the error via the receive StatusInfo written in the Status array and the IntStatus interrupt status register.

The receive process can generate several types of errors: AlignmentError, RangeError, LengthError, SymbolError, CRCError, Overrun, and NoDescriptor. All have corresponding bits in the receive StatusInfo. In addition to the separate bits in the StatusInfo, AlignmentError, RangeError, LengthError, SymbolError, and CRCError are ORed together into the Error bit of the StatusInfo. Errors are also propagated to the IntStatus register; the RxError bit in the IntStatus register is set if there is an AlignmentError, RangeError, LengthError, SymbolError, CRCError, or NoDescriptor error; nonfatal overrun errors are reported in the RxError bit of the IntStatus register; fatal Overrun errors are report in the RxOverrun bit of the IntStatus register. On fatal overrun errors, the Rx data path needs to be soft reset by setting the RxReset bit in the Command register.

Overrun errors can have three causes:

- In the case of a multi-fragment reception, the next descriptor may be missing. In this case the NoDescriptor field is set in the status word of the previous descriptor and the RxError in the IntStatus register is set. This error is nonfatal.
- The data flow on the receiver data interface stalls, corrupting the packet. In this case the overrun bit in the status word is set and the RxError bit in the IntStatus register is set. This error is nonfatal.
- The flow of reception statuses stalls and a new status has to be written while a previous status still waits to be transferred across the memory interface. This error will corrupt the hardware state and requires the hardware to be soft reset. The error is detected and sets the Overrun bit in the IntStatus register.

The first overrun situation will result in an incomplete frame with a NoDescriptor status and the RxError bit in IntStatus set. Software should discard the partially received frame. In the second overrun situation the frame data will be corrupt which results in the Overrun status bit being set in the Status word while the IntError interrupt bit is set. In the third case

receive errors cannot be reported in the receiver Status arrays which corrupts the hardware state; the errors will still be reported in the IntStatus register's Overrun bit. The RxReset bit in the Command register should be used to soft reset the hardware.

Device drivers should catch the above receive errors and take action.

Receive triggers interrupts

The receive data path can generate four different interrupt types:

- If the Interrupt bit in the descriptor Control field is set, the Rx DMA will set the RxDoneInt bit in the IntStatus register after receiving a fragment and committing the associated data and status to memory. Even if a descriptor (fragment) is not the last in a multi-fragment frame, the Interrupt bit in the descriptor can be used to generate an interrupt.
- If the descriptor array is full while the Ethernet hardware is enabled, the hardware will set the RxFinishedInt bit of the IntStatus register.
- If the AHB interface does not consume receive statuses at a sufficiently high bandwidth, the receive status process may overrun, in which case the RxOverrun bit will be set in the IntStatus register.
- If there is a receive error (AlignmentError, RangeError, LengthError, SymbolError, or CRCErrror), or a multi-fragment frame where the device driver did provide descriptors for the initial fragments but did not provide the descriptors for the rest of the fragments, or if a nonfatal data Overrun occurred, the hardware will set the RxErrorInt bit of the IntStatus register.

All of the above interrupts can be enabled and disabled by setting or resetting the corresponding bits in the IntEnable register. Enabling or disabling does not affect the IntStatus register contents, only the propagation of the interrupt status to the CPU (via the NVIC).

The interrupts, either of individual frames or of the whole list, are a good means of communication between the DMA manager and the device driver, triggering the device driver to inspect the status words of descriptors that have been processed.

Device driver processes receive data

As a response to status (e.g. RxDoneInt) interrupts or polling of the RxProduceIndex, the device driver can read the descriptors that have been handed over to it by the hardware (RxProduceIndex - RxConsumeIndex). The device driver should inspect the status words in the status array to check for multi-fragment receptions and receive errors.

The device driver can forward receive data and status to upper software layers. After processing of data and status, the descriptors, statuses and data buffers may be recycled and handed back to hardware by incrementing the RxConsumeIndex.

Receive example

[Figure 23](#) illustrates the receive process in an example receiving a frame of 19 bytes.

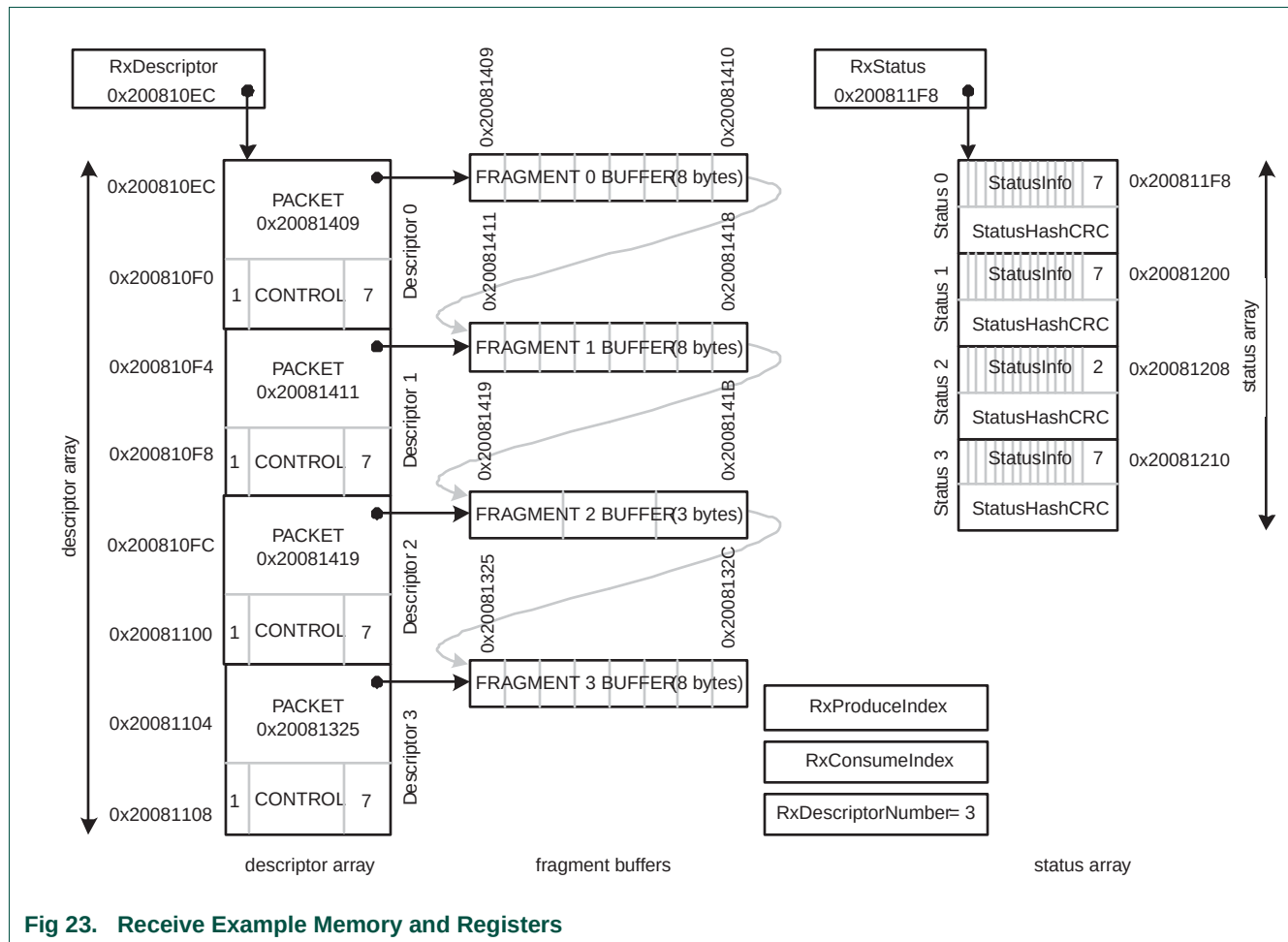


Fig 23. Receive Example Memory and Registers

After reset, the values of the DMA registers will be zero. During initialization, the device driver will allocate the descriptor and status array in memory. In this example, an array of four descriptors is allocated; the array is 4x2x4 bytes and aligned on a 4 byte address boundary. Since the number of descriptors matches the number of statuses, the status array consists of four elements; the array is 4x2x4 bytes and aligned on a 8 byte address boundary. The device driver writes the base address of the descriptor array (0x2008 10EC) in the RxDescriptor register, and the base address of the status array (0x2008 11F8) in the RxStatus register. The device driver writes the number of descriptors and statuses minus 1 (3) in the RxDescriptorNumber register. The descriptors and statuses in the arrays need not be initialized yet.

After allocating the descriptors, a fragment buffer needs to be allocated for each of the descriptors. Each fragment buffer can be between 1 byte and 2 k bytes. The base address of the fragment buffer is stored in the Packet field of the descriptors. The number of bytes in the fragment buffer is stored in the Size field of the descriptor Control word. The Interrupt field in the Control word of the descriptor can be set to generate an interrupt as soon as the descriptor has been filled by the receive process. In this example the fragment buffers are 8 bytes, so the value of the Size field in the Control word of the descriptor is set to 7. Note that in this example, the fragment buffers are actually a

continuous memory space; even when a frame is distributed over multiple fragments it will typically be in a linear, continuous memory space; when the descriptors wrap at the end of the descriptor array the frame will not be in a continuous memory space.

The device driver should enable the receive process by writing a 1 to the RxEnable bit of the Command register, after which the MAC needs to be enabled by writing a 1 to the 'RECEIVE ENABLE' bit of the MAC1 configuration register. The Ethernet block will now start receiving Ethernet frames. To reduce the processor interrupt load, some interrupts can be disabled by setting the relevant bits in the IntEnable register.

After the Rx DMA manager is enabled, it will start issuing descriptor read commands. In this example the number of descriptors is 4. Initially the RxProduceIndex and RxConsumeIndex are 0. Since the descriptor array is considered full if $\text{RxProduceIndex} == \text{RxConsumeIndex} - 1$, the Rx DMA manager can only read ($\text{RxConsumeIndex} - \text{RxProduceIndex} - 1$) 3 descriptors; note the wrapping.

After enabling the receive function in the MAC, data reception will begin starting at the next frame i.e. if the receive function is enabled while the RMII interface is halfway through receiving a frame, the frame will be discarded and reception will start at the next frame. The Ethernet block will strip the preamble and start of frame delimiter from the frame. If the frame passes the receive filtering, the Rx DMA manager will start writing the frame to the first fragment buffer.

Suppose the frame is 19 bytes long. Due to the buffer sizes specified in this example, the frame will be distributed over three fragment buffers. After writing the initial 8 bytes in the first fragment buffer, the status for the first fragment buffer will be written and the Rx DMA will continue filling the second fragment buffer. Since this is a multi-fragment receive, the status of the first fragment will have a 0 for the LastFrag bit in the StatusInfo word; the RxSize field will be set to 7 (8, -1 encoded). After writing the 8 bytes in the second fragment the Rx DMA will continue writing the third fragment. The status of the second fragment will be like the status of the first fragment: LastFrag = 0, RxSize = 7. After writing the three bytes in the third fragment buffer, the end of the frame has been reached and the status of the third fragment is written. The third fragment's status will have the LastFrag bit set to 1 and the RxSize equal to 2 (3, -1 encoded).

The next frame received from the RMII interface will be written to the fourth fragment buffer i.e. five bytes of the third buffer will be unused.

The Rx DMA manager uses an internal tag protocol in the memory interface to check that the receive data and status have been committed to memory. After the status of the fragments are committed to memory, an RxDoneInt interrupt will be triggered, which activates the device driver to inspect the status information. In this example, all descriptors have the Interrupt bit set in the Control word i.e. all descriptors will generate an interrupt after committing data and status to memory.

In this example the receive function cannot read new descriptors as long as the device driver does not increment the RxConsumeIndex, because the descriptor array is full (even though one descriptor is not programmed yet). Only after the device driver has forwarded the receive data to application software, and after the device driver has updated the RxConsumeIndex by incrementing it, will the Ethernet block can continue reading descriptors and receive data. The device driver will probably increment the RxConsumeIndex by 3, since the driver will forward the complete frame consisting of three fragments to the application, and hence free up three descriptors at the same time.

Each four pairs of bits transferred on the RMII interface are transferred as a byte on the data write interface after being delayed by 128 or 136 cycles for filtering by the receive filter and buffer modules. The Ethernet block removes preamble, frame start delimiter, and CRC from the data and checks the CRC. To limit the buffer NoDescriptor error probability, three descriptors are buffered. The value of the RxProduceIndex is only updated after status information has been committed to memory, which is checked by an internal tag protocol in the memory interface. The software device driver will process the receive data, after which the device driver will update the RxConsumeIndex.

10.17.5 Transmission retry

If a collision on the Ethernet occurs, it usually takes place during the collision window spanning the first 64 bytes of a frame. If collision is detected, the Ethernet block will retry the transmission. For this purpose, the first 64 bytes of a frame are buffered, so that this data can be used during the retry. A transmission retry within the first 64 bytes in a frame is fully transparent to the application and device driver software.

When a collision occurs outside of the 64 byte collision window, a LateCollision error is triggered, and the transmission is aborted. After a LateCollision error, the remaining data in the transmit frame will be discarded. The Ethernet block will set the Error and LateCollision bits in the frame's status fields. The TxError bit in the IntStatus register will be set. If the corresponding bit in the IntEnable register is set, the TxError bit in the IntStatus register will be propagated to the CPU (via the NVIC). The device driver software should catch the interrupt and take appropriate actions.

The 'RETRANSMISSION MAXIMUM' field of the CLRT register can be used to configure the maximum number of retries before aborting the transmission.

10.17.6 Status hash CRC calculations

For each received frame, the Ethernet block is able to detect the destination address and source address and from them calculate the corresponding hash CRCs. To perform the computation, the Ethernet block features two internal blocks: one is a controller synchronized with the beginning and the end of each frame, the second block is the CRC calculator.

When a new frame is detected, internal signaling notifies the controller. The controller starts counting the incoming bytes of the frame, which correspond to the destination address bytes. When the sixth (and last) byte is counted, the controller notifies the calculator to store the corresponding 32-bit CRC into a first inner register. Then the controller repeats counting the next incoming bytes, in order to get synchronized with the source address. When the last byte of the source address is encountered, the controller again notifies the CRC calculator, which freezes until the next new frame. When the calculator receives this second notification, it stores the present 32-bit CRC into a second inner register. Then the CRCs remain frozen in their own registers until new notifications arise.

The destination address and source address hash CRCs being written in the StatusHashCRC word are the nine most significant bits of the 32-bit CRCs as calculated by the CRC calculator.

10.17.7 Duplex modes

The Ethernet block can operate in full duplex and half duplex mode. Half or full duplex mode needs to be configured by the device driver software during initialization.

For a full duplex connection the FullDuplex bit of the Command register needs to be set to 1 and the FULL-DUPLEX bit of the MAC2 configuration register needs to be set to 1; for half duplex the same bits need to be set to 0.

10.17.8 IEEE 802.3/Clause 31 flow control

Overview

For full duplex connections, the Ethernet block supports IEEE 802.3/clause 31 flow control using pause frames. This type of flow control may be used in full-duplex point-to-point connections. Flow control allows a receiver to stall a transmitter e.g. when the receive buffers are (almost) full. For this purpose, the receiving side sends a pause frame to the transmitting side.

Pause frames use units of 512 bit times corresponding to 128 rx_clk/tx_clk cycles.

Receive flow control

In full-duplex mode, the Ethernet block will suspend its transmissions when it receives a pause frame. Rx flow control is initiated by the receiving side of the transmission. It is enabled by setting the 'RX FLOW CONTROL' bit in the MAC1 configuration register. If the RX FLOW CONTROL' bit is zero, then the Ethernet block ignores received pause control frames. When a pause frame is received on the Rx side of the Ethernet block, transmission on the Tx side will be interrupted after the currently transmitting frame has completed, for an amount of time as indicated in the received pause frame. The transmit data path will stop transmitting data for the number of 512 bit slot times encoded in the pause-timer field of the received pause control frame.

By default the received pause control frames are not forwarded to the device driver. To forward the receive flow control frames to the device driver, set the 'PASS ALL RECEIVE FRAMES' bit in the MAC1 configuration register.

Transmit flow control

If case device drivers need to stall the receive data e.g. because software buffers are full, the Ethernet block can transmit pause control frames. Transmit flow control needs to be initiated by the device driver software; there is no IEEE 802.3/31 flow control initiated by hardware, such as the DMA managers.

With software flow control, the device driver can detect a situation in which the process of receiving frames needs to be interrupted by sending out Tx pause frames. Note that due to Ethernet delays, a few frames can still be received before the flow control takes effect and the receive stream stops.

Transmit flow control is activated by writing 1 to the TxFlowControl bit of the Command register. When the Ethernet block operates in full duplex mode, this will result in transmission of IEEE 802.3/31 pause frames. The flow control continues until a 0 is written to TxFlowControl bit of the Command register.

If the MAC is operating in full-duplex mode, then setting the TxFlowControl bit of the Command register will start a pause frame transmission. The value inserted into the pause-timer value field of transmitted pause frames is programmed via the PauseTimer[15:0] bits in the FlowControlCounter register. When the TxFlowControl bit is de-asserted, another pause frame having a pause-timer value of 0x0000 is automatically sent to abort flow control and resume transmission.

When flow control be in force for an extended time, a sequence of pause frames must be transmitted. This is supported with a mirror counter mechanism. To enable mirror counting, a nonzero value is written to the MirrorCounter[15:0] bits in the FlowControlCounter register. When the TxFlowControl bit is asserted, a pause frame is transmitted. After sending the pause frame, an internal mirror counter is initialized to zero. The internal mirror counter starts incrementing one every 512 bit-slot times. When the internal mirror counter reaches the MirrorCounter value, another pause frame is transmitted with pause-timer value equal to the PauseTimer field from the FlowControlCounter register, the internal mirror counter is reset to zero and restarts counting. The register MirrorCounter[15:0] is usually set to a smaller value than register PauseTimer[15:0] to ensure an early expiration of the mirror counter, allowing time to send a new pause frame before the transmission on the other side can resume. By continuing to send pause frames before the transmitting side finishes counting the pause timer, the pause can be extended as long as TxFlowControl is asserted. This continues until TxFlowControl is de-asserted when a final pause frame having a pause-timer value of 0x0000 is automatically sent to abort flow control and resume transmission. To disable the mirror counter mechanism, write the value 0 to MirrorCounter field in the FlowControlCounter register. When using the mirror counter mechanism, account for time-of-flight delays, frame transmission time, queuing delays, crystal frequency tolerances, and response time delays by programming the MirrorCounter conservatively, typically about 80% of the PauseTimer value.

If the software device driver sets the MirrorCounter field of the FlowControlCounter register to zero, the Ethernet block will only send one pause control frame. After sending the pause frame an internal pause counter is initialized at zero; the internal pause counter is incremented by one every 512 bit-slot times. Once the internal pause counter reaches the value of the PauseTimer register, the TxFlowControl bit in the Command register will be reset. The software device driver can poll the TxFlowControl bit to detect when the pause completes.

The value of the internal counter in the flow control module can be read out via the FlowControlStatus register. If the MirrorCounter is nonzero, the FlowControlStatus register will return the value of the internal mirror counter; if the MirrorCounter is zero the FlowControlStatus register will return the value of the internal pause counter value.

The device driver is allowed to dynamically modify the MirrorCounter register value and switch between zero MirrorCounter and nonzero MirrorCounter modes.

Transmit flow control is enabled via the 'TX FLOW CONTROL' bit in the MAC1 configuration register. If the 'TX FLOW CONTROL' bit is zero, then the MAC will not transmit pause control frames, software must not initiate pause frame transmissions, and the TxFlowControl bit in the Command register should be zero.

Transmit flow control example

[Figure 24](#) illustrates the transmit flow control.

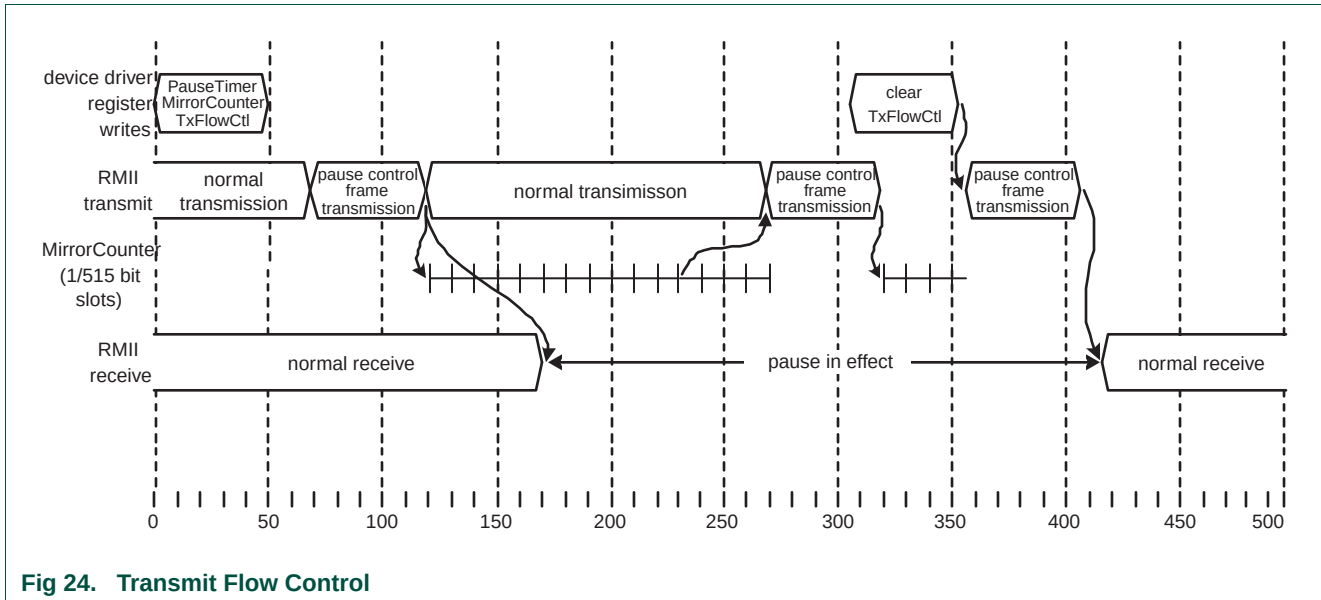


Fig 24. Transmit Flow Control

In this example, a frame is received while transmitting another frame (full duplex.) The device driver detects that some buffer might overrun and enables the transmit flow control by programming the PauseTimer and MirrorCounter fields of the FlowControlCounter register, after which it enables the transmit flow control by setting the TxFlowControl bit in the Command register.

As a response to the enabling of the flow control a pause control frame will be sent after the currently transmitting frame has been transmitted. When the pause frame transmission completes the internal mirror counter will start counting bit slots; as soon as the counter reaches the value in the MirrorCounter field another pause frame is transmitted. While counting the transmit data path will continue normal transmissions.

As soon as software disables transmit flow control a zero pause control frame is transmitted to resume the receive process.

10.17.9 Half-Duplex mode backpressure

When in half-duplex mode, backpressure can be generated to stall receive packets by sending continuous preamble that basically jams any other transmissions on the Ethernet medium. When the Ethernet block operates in half duplex mode, asserting the TxFlowControl bit in the Command register will result in applying continuous preamble on the Ethernet wire, effectively blocking traffic from any other Ethernet station on the same segment.

In half duplex mode, when the TxFlowControl bit goes high, continuous preamble is sent until TxFlowControl is de-asserted. If the medium is idle, the Ethernet block begins transmitting preamble, which raises carrier sense causing all other stations to defer. In the event the transmitting of preamble causes a collision, the backpressure 'rides through' the collision. The colliding station backs off and then defers to the backpressure. If during backpressure, the user wishes to send a frame, the backpressure is interrupted, the frame sent and then the backpressure resumed. If TxFlowControl is asserted for longer than 3.3 ms in 10 Mbps mode or 0.33 ms in 100 Mbps mode, backpressure will cease sending preamble for several byte times to avoid the jabber limit.

10.17.10 Receive filtering

Features of receive filtering

The Ethernet MAC has several receive packet filtering functions that can be configured from the software driver:

- Perfect address filter: allows packets with a perfectly matching station address to be identified and passed to the software driver.
- Hash table filter: allows imperfect filtering of packets based on the station address.
- Unicast/multicast/broadcast filtering: allows passing of all unicast, multicast, and/or broadcast packets.
- Magic packet filter: detection of magic packets to generate a Wake-on-LAN interrupt.

The filtering functions can be logically combined to create complex filtering functions. Furthermore, the Ethernet block can pass or reject runt packets smaller than 64 bytes; a promiscuous mode allows all packets to be passed to software.

Overview

The Ethernet block has the capability to filter out receive frames by analyzing the Ethernet destination address in the frame. This capability greatly reduces the load on the host system, because Ethernet frames that are addressed to other stations would otherwise need to be inspected and rejected by the device driver software, using up bandwidth, memory space, and host CPU time. Address filtering can be implemented using the perfect address filter or the (imperfect) hash filter. The latter produces a 6-bit hash code which can be used as an index into a 64 entry programmable hash table. [Figure 25](#) depicts a functional view of the receive filter.

At the top of the diagram the Ethernet receive frame enters the filters. Each filter is controlled by signals from control registers; each filter produces a 'Ready' output and a 'Match' output. If 'Ready' is 0 then the Match value is 'don't care'; if a filter finishes filtering then it will assert its Ready output; if the filter finds a matching frame it will assert the Match output along with the Ready output. The results of the filters are combined by logic functions into a single RxAbort output. If the RxAbort output is asserted, the frame does not need to be received.

In order to reduce memory traffic, the receive data path has a buffer of 68 bytes. The Ethernet MAC will only start writing a frame to memory after 68 byte delays. If the RxAbort signal is asserted during the initial 68 bytes of the frame, the frame can be discarded and removed from the buffer and not stored to memory at all, not using up receive descriptors, etc. If the RxAbort signal is asserted after the initial 68 bytes in a frame (probably due to reception of a Magic Packet), part of the frame is already written to memory and the Ethernet MAC will stop writing further data in the frame to memory; the FailFilter bit in the status word of the frame will be set to indicate that the software device driver can discard the frame immediately.

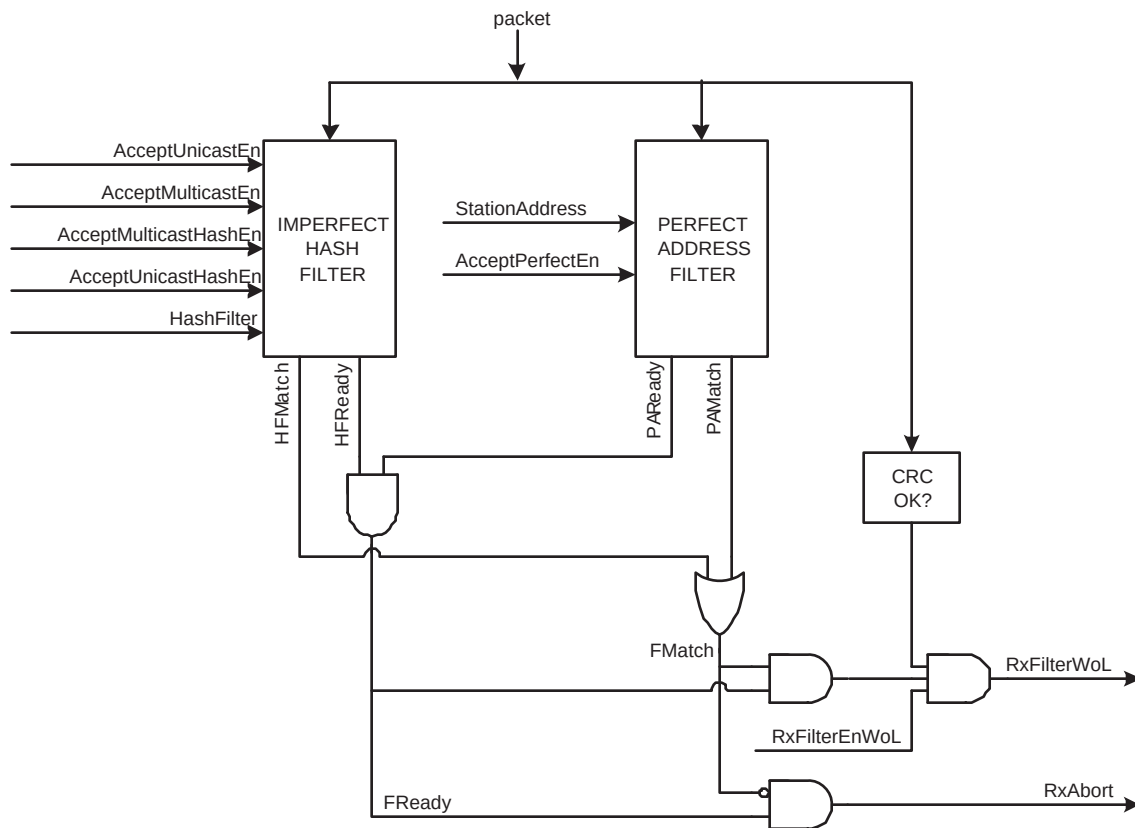


Fig 25. Receive filter block diagram

Unicast, broadcast and multicast

Generic filtering based on the type of frame (unicast, multicast or broadcast) can be programmed using the AcceptUnicastEn, AcceptMulticastEn, or AcceptBroadcastEn bits of the RxFilterCtrl register. Setting the AcceptUnicast, AcceptMulticast, and AcceptBroadcast bits causes all frames of types unicast, multicast and broadcast, respectively, to be accepted, ignoring the Ethernet destination address in the frame. To program promiscuous mode, i.e. to accept all frames, set all 3 bits to 1.

Perfect address match

When a frame with a unicast destination address is received, a perfect filter compares the destination address with the 6 byte station address programmed in the station address registers SA0, SA1, SA2. If the AcceptPerfectEn bit in the RxFilterCtrl register is set to 1, and the address matches, the frame is accepted.

Imperfect hash filtering

An imperfect filter is available, based on a hash mechanism. This filter applies a hash function to the destination address and uses the hash to access a table that indicates if the frame should be accepted. The advantage of this type of filter is that a small table can cover any possible address. The disadvantage is that the filtering is imperfect, i.e. sometimes frames are accepted that should have been discarded.

- Hash function:
 - The standard Ethernet cyclic redundancy check (CRC) function is calculated from the 6 byte destination address in the Ethernet frame (this CRC is calculated anyway as part of calculating the CRC of the whole frame), then bits [28:23] out of the 32-bit CRC result are taken to form the hash. The 6-bit hash is used to access the hash table: it is used as an index in the 64-bit HashFilter register that has been programmed with accept values. If the selected accept value is 1, the frame is accepted.
 - The device driver can initialize the hash filter table by writing to the registers HashFilterL and HashfilterH. HashFilterL contains bits 0 through 31 of the table and HashFilterH contains bit 32 through 63 of the table. So, hash value 0 corresponds to bit 0 of the HashfilterL register and hash value 63 corresponds to bit 31 of the HashFilterH register.
- Multicast and unicast
 - The imperfect hash filter can be applied to multicast addresses, by setting the AcceptMulticastHashEn bit in the RxFilter register to 1.
 - The same imperfect hash filter that is available for multicast addresses can also be used for unicast addresses. This is useful to be able to respond to a multitude of unicast addresses without enabling all unicast addresses. The hash filter can be applied to unicast addresses by setting the AcceptUnicastHashEn bit in the RxFilter register to 1.

Enabling and disabling filtering

The filters as defined in the sections above can be bypassed by setting the PassRxFilter bit in the Command register. When the PassRxFilter bit is set, all receive frames will be passed to memory. In this case the device driver software has to implement all filtering functionality in software. Setting the PassRxFilter bit does not affect the runt frame filtering as defined in the next section.

Runt frames

A frame with less than 64 bytes (or 68 bytes for VLAN frames) is shorter than the minimum Ethernet frame size and therefore considered erroneous; they might be collision fragments. The receive data path automatically filters and discards these runt frames without writing them to memory and using a receive descriptor.

When a runt frame has a correct CRC there is a possibility that it is intended to be useful. The device driver can receive the runt frames with correct CRC by setting the PassRuntFrame bit of the Command register to 1.

10.17.11 Power management

The Ethernet block supports power management by means of clock switching. All clocks in the Ethernet core can be switched off. If Wake-up on LAN is needed, the rx_clk should not be switched off.

10.17.12 Wake-up on LAN

Overview

The Ethernet block supports power management with remote wake-up over LAN. The host system can be powered down, even including part of the Ethernet block itself, while the Ethernet block continues to listen to packets on the LAN. Appropriately formed packets can be received and recognized by the Ethernet block and used to trigger the host system to wake up from its power-down state.

Wake-up of the system takes effect through an interrupt. When a wake-up event is detected, the WakeupInt bit in the IntStatus register is set. The interrupt status will trigger an interrupt if the corresponding WakeupIntEn bit in the IntEnable register is set. This interrupt should be used by system power management logic to wake up the system.

While in a power-down state the packet that generates a Wake-up on LAN event is lost.

There are two ways in which Ethernet packets can trigger wake-up events: generic Wake-up on LAN and Magic Packet. Magic Packet filtering uses an additional filter for Magic Packet detection. In both cases a Wake-up on LAN event is only triggered if the triggering packet has a valid CRC. [Figure 25](#) shows the generation of the wake-up signal.

The RxFilterWoLStatus register can be read by the software to inspect the reason for a Wake-up event. Before going to power-down the power management software should clear the register by writing the RxFilterWoLClear register.

NOTE: when entering in power-down mode, a receive frame might be not entirely stored into the Rx buffer. In this situation, after turning exiting power-down mode, the next receive frame is corrupted due to the data of the previous frame being added in front of the last received frame. Software drivers have to reset the receive data path just after exiting power-down mode.

The following subsections describe the two Wake-up on LAN mechanisms.

Filtering for WoL

The receive filter functionality can be used to generate Wake-up on LAN events. If the RxFilterEnWoL bit of the RxFilterCtrl register is set, the receive filter will set the WakeupInt bit of the IntStatus register if a frame is received that passes the filter. The interrupt will only be generated if the CRC of the frame is correct.

Magic Packet WoL

The Ethernet block supports wake-up using Magic Packet technology (see 'Magic Packet technology', Advanced Micro Devices). A Magic Packet is a specially formed packet solely intended for wake-up purposes. This packet can be received, analyzed and recognized by the Ethernet block and used to trigger a wake-up event.

A Magic Packet is a packet that contains in its data portion the station address repeated 16 times with no breaks or interruptions, preceded by 6 Magic Packet synchronization bytes with the value 0xFF. Other data may be surrounding the Magic Packet pattern in the data portion of the packet. The whole packet must be a well-formed Ethernet frame.

The magic packet detection unit analyzes the Ethernet packets, extracts the packet address and checks the payload for the Magic Packet pattern. The address from the packet is used for matching the pattern (not the address in the SA0/1/2 registers.) A magic packet only sets the wake-up interrupt status bit if the packet passes the receive filter as illustrated in [Figure 25](#): the result of the receive filter is ANDed with the magic packet filter result to produce the result.

Magic Packet filtering is enabled by setting the MagicPacketEnWoL bit of the RxFilterCtrl register. Note that when doing Magic Packet WoL, the RxFilterEnWoL bit in the RxFilterCtrl register should be 0. Setting the RxFilterEnWoL bit to 1 would accept all packets for a matching address, not just the Magic Packets i.e. WoL using Magic Packets is more strict.

When a magic packet is detected, apart from the WakeupInt bit in the IntStatus register, the MagicPacketWoL bit is set in the RxFilterWoLStatus register. Software can reset the bit writing a 1 to the corresponding bit of the RxFilterWoLClear register.

Example: An example of a Magic Packet with station address 0x11 0x22 0x33 0x44 0x55 0x66 is the following (MISC indicates miscellaneous additional data bytes in the packet):

[illegible]

10.17.13 Enabling and disabling receive and transmit

Enabling and disabling reception

After reset, the receive function of the Ethernet block is disabled. The receive function can be enabled by the device driver setting the RxEnable bit in the Command register and the "RECEIVE ENABLE" bit in the MAC1 configuration register (in that order).

The status of the receive data path can be monitored by the device driver by reading the RxStatus bit of the Status register. [Figure 26](#) illustrates the state machine for the generation of the RxStatus bit.

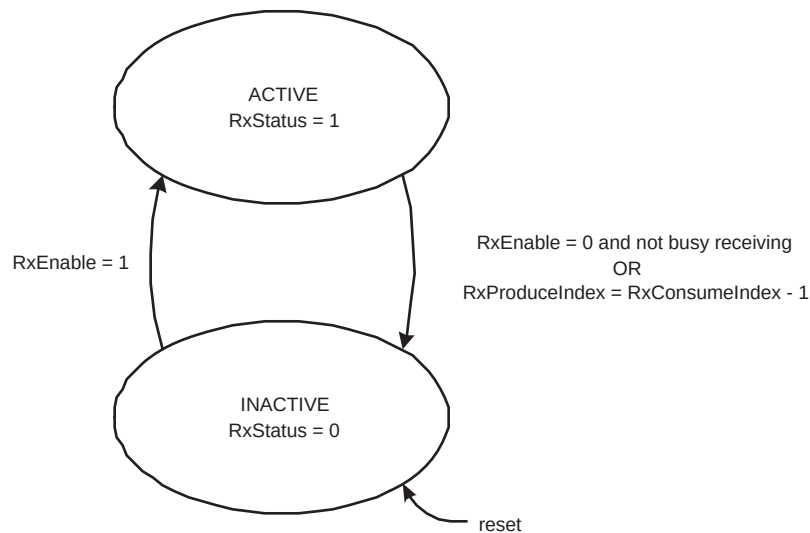


Fig 26. Receive Active/Inactive state machine

After a reset, the state machine is in the INACTIVE state. As soon as the RxEnable bit is set in the Command register, the state machine transitions to the ACTIVE state. As soon as the RxEnable bit is cleared, the state machine returns to the INACTIVE state. If the receive data path is busy receiving a packet while the receive data path gets disabled, the packet will be received completely, stored to memory along with its status before returning to the INACTIVE state. Also if the Receive descriptor array is full, the state machine will return to the INACTIVE state.

For the state machine in [Figure 26](#), a soft reset is like a hardware reset assertion, i.e. after a soft reset the receive data path is inactive until the data path is re-enabled.

Enabling and disabling transmission

After reset, the transmit function of the Ethernet block is disabled. The Tx transmit data path can be enabled by the device driver setting the TxEnable bit in the Command register to 1.

The status of the transmit data paths can be monitored by the device driver reading the TxStatus bit of the Status register. [Figure 27](#) illustrates the state machine for the generation of the TxStatus bit.

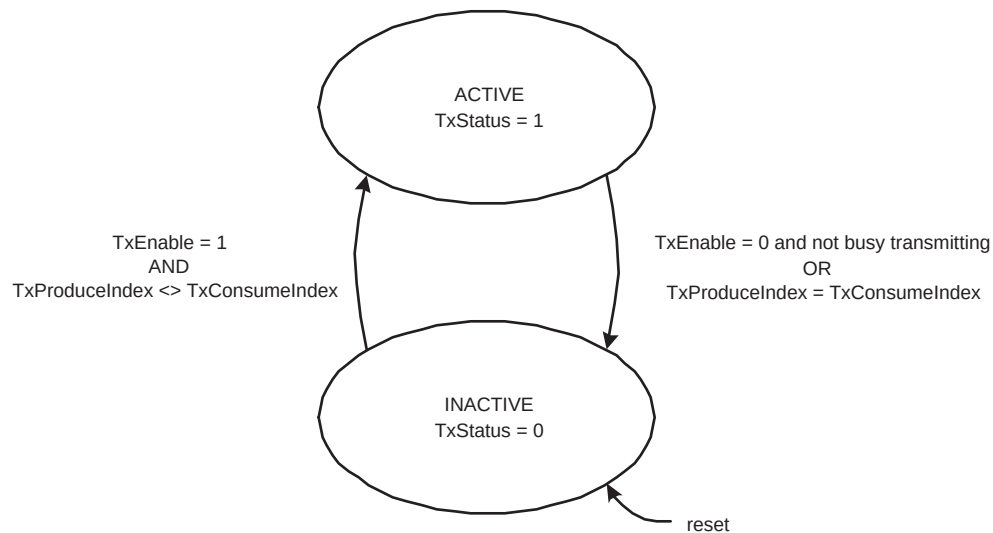


Fig 27. Transmit Active/Inactive state machine

After reset, the state machine is in the INACTIVE state. As soon as the TxEnable bit is set in the Command register and the Produce and Consume indices are not equal, the state machine transitions to the ACTIVE state. As soon as the TxEnable bit is cleared and the transmit data path has completed all pending transmissions, including committing the transmission status to memory, the state machine returns to the INACTIVE state. The state machine will also return to the INACTIVE state if the Produce and Consume indices are equal again i.e. all frames have been transmitted.

For the state machine in [Figure 27](#), a soft reset is like a hardware reset assertion, i.e. after a soft reset the transmit data path is inactive until the data path is re-enabled.

10.17.14 Transmission padding and CRC

In the case of a frame of less than 60 bytes (or 64 bytes for VLAN frames), the Ethernet block can pad the frame to 64 or 68 bytes including a 4 bytes CRC Frame Check Sequence (FCS). Padding is affected by the value of the 'AUTO DETECT PAD ENABLE' (ADPEN), 'VLAN PAD ENABLE' (VLPEN) and 'PAD/CRC ENABLE' (PADEN) bits of the MAC2 configuration register, as well as the Override and Pad bits from the transmit descriptor Control word. CRC generation is affected by the 'CRC ENABLE' (CRCE) and 'DELAYED CRC' (DCRC) bits of the MAC2 configuration register, and the Override and CRC bits from the transmit descriptor Control word.

The effective pad enable (EPADEN) is equal to the 'PAD/CRC ENABLE' bit from the MAC2 register if the Override bit in the descriptor is 0. If the Override bit is 1, then EPADEN will be taken from the descriptor Pad bit. Likewise the effective CRC enable (ECRCE) equals CRCE if the Override bit is 0, otherwise it equal the CRC bit from the descriptor.

If padding is required and enabled, a CRC will always be appended to the padded frames. A CRC will only be appended to the non-padded frames if ECRCE is set.

If EPADEN is 0, the frame will not be padded and no CRC will be added unless ECRCE is set.

If EPADEN is 1, then small frames will be padded and a CRC will always be added to the padded frames. In this case if ADPEN and VLPEN are both 0, then the frames will be padded to 60 bytes and a CRC will be added creating 64 bytes frames; if VLPEN is 1, the frames will be padded to 64 bytes and a CRC will be added creating 68 bytes frames; if ADPEN is 1, while VLPEN is 0 VLAN frames will be padded to 64 bytes, non VLAN frames will be padded to 60 bytes, and a CRC will be added to padded frames, creating 64 or 68 bytes padded frames.

If CRC generation is enabled, CRC generation can be delayed by four bytes by setting the DELAYED CRC bit in the MAC2 register, in order to skip proprietary header information.

10.17.15 Huge frames and frame length checking

The 'HUGE FRAME ENABLE' bit in the MAC2 configuration register can be set to 1 to enable transmission and reception of frames of any length. Huge frame transmission can be enabled on a per frame basis by setting the Override and Huge bits in the transmit descriptor Control word.

When enabling huge frames, the Ethernet block will not check frame lengths and report frame length errors (RangeError and LengthError). If huge frames are enabled, the received byte count in the RSV register may be invalid because the frame may exceed the maximum size; the RxSize fields from the receive status arrays will be valid.

Frame lengths are checked by comparing the length/type field of the frame to the actual number of bytes in the frame. A LengthError is reported by setting the corresponding bit in the receive StatusInfo word.

The MAXF register allows the device driver to specify the maximum number of bytes in a frame. The Ethernet block will compare the actual receive frame to the MAXF value and report a RangeError in the receive StatusInfo word if the frame is larger.

10.17.16 Statistics counters

Generally, Ethernet applications maintain many counters that track Ethernet traffic statistics. There are a number of standards specifying such counters, such as IEEE std 802.3 / clause 30. Other standards are RFC 2665 and RFC 2233.

The approach taken here is that by default all counters are implemented in software. With the help of the StatusInfo field in frame statuses, many of the important statistics events listed in the standards can be counted by software.

10.17.17 MAC status vectors

Transmit and receive status information as detected by the MAC are available in registers TSV0, TSV1 and RSV so that software can poll them. These registers are normally of limited use because the communication between driver software and the Ethernet block takes place primarily through frame descriptors. Statistical events can be counted by software in the device driver. However, for debug purposes the transmit and receive status vectors are made visible. They are valid as long as the internal status of the MAC is valid and should typically only be read when the transmit and receive processes are halted.

10.17.18 Reset

The Ethernet block has a hard reset input which is connected to the chip reset, as well as several soft resets which can be activated by setting the appropriate bit(s) in registers. All registers in the Ethernet block have a value of 0 after a hard reset, unless otherwise specified.

Hard reset

After a hard reset, all registers will be set to their default value.

Soft reset

Parts of the Ethernet block can be soft reset by setting bits in the Command register and the MAC1 configuration register. The MAC1 register has six different reset bits:

- **SOFT RESET:** Setting this bit will put all modules in the MAC in reset, except for the MAC registers (at addresses 0x000 to 0x0FC). The value of the soft reset after a hardware reset assertion is 1, i.e. the soft reset needs to be cleared after a hardware reset.
- **SIMULATION RESET:** Resets the random number generator in the Transmit Function. The value after a hardware reset assertion is 0.
- **RESET MCS/Rx:** Setting this bit will reset the MAC Control Sublayer (pause frame logic) and the receive function in the MAC. The value after a hardware reset assertion is 0.
- **RESET Rx:** Setting this bit will reset the receive function in the MAC. The value after a hardware reset assertion is 0.
- **RESET MCS/Tx:** Setting this bit will reset the MAC Control Sublayer (pause frame logic) and the transmit function in the MAC. The value after a hardware reset assertion is 0.
- **RESET Tx:** Setting this bit will reset the transmit function of the MAC. The value after a hardware reset assertion is 0.

The above reset bits must be cleared by software.

The Command register has three different reset bits:

- **TxReset:** Writing a '1' to the TxReset bit will reset the transmit data path, excluding the MAC portions, including all (read-only) registers in the transmit data path, as well as the TxProduceIndex register in the host registers module. A soft reset of the transmit data path will abort all AHB transactions of the transmit data path. The reset bit will be cleared autonomously by the Ethernet block. A soft reset of the Tx data path will clear the TxStatus bit in the Status register.
- **RxReset:** Writing a '1' to the RxReset bit will reset the receive data path, excluding the MAC portions, including all (read-only) registers in the receive data path, as well as the RxConsumeIndex register in the host registers module. A soft reset of the receive data path will abort all AHB transactions of the receive data path. The reset bit will be cleared autonomously by the Ethernet block. A soft reset of the Rx data path will clear the RxStatus bit in the Status register.

- RegReset: Resets all of the data paths and registers in the host registers module, excluding the registers in the MAC. A soft reset of the registers will also abort all AHB transactions of the transmit and receive data path. The reset bit will be cleared autonomously by the Ethernet block.

To do a full soft reset of the Ethernet block, device driver software must:

- Set the 'SOFT RESET' bit in the MAC1 register to 1.
- Set the RegReset bit in the Command register, this bit clears automatically.
- Re-initialize the MAC registers (0x000 to 0x0FC).
- Reset the 'SOFT RESET' bit in the MAC1 register to 0.

To reset just the transmit data path, the device driver software has to:

- Set the 'RESET MCS/Tx' bit in the MAC1 register to 1.
- Disable the Tx DMA managers by setting the TxEnable bits in the Command register to 0.
- Set the TxReset bit in the Command register, this bit clears automatically.
- Reset the 'RESET MCS/Tx' bit in the MAC1 register to 0.

To reset just the receive data path, the device driver software has to:

- Disable the receive function by resetting the 'RECEIVE ENABLE' bit in the MAC1 configuration register and resetting of the RxEnable bit of the Command register.
- Set the 'RESET MCS/Rx' bit in the MAC1 register to 1.
- Set the RxReset bit in the Command register, this bit clears automatically.
- Reset the 'RESET MCS/Rx' bit in the MAC1 register to 0.

10.17.19 Ethernet errors

The Ethernet block generates errors for the following conditions:

- A reception can cause an error: AlignmentError, RangeError, LengthError, SymbolError, CRCError, NoDescriptor, or Overrun. These are reported back in the receive StatusInfo and in the interrupt status register (IntStatus).
- A transmission can cause an error: LateCollision, ExcessiveCollision, ExcessiveDefer, NoDescriptor, or Underrun. These are reported back in the transmission StatusInfo and in the interrupt status register (IntStatus).

10.18 AHB bandwidth

The Ethernet block is connected to an AHB bus which must carry all of the data and control information associated with all Ethernet traffic in addition to the CPU accesses required to operate the Ethernet block and deal with message contents.

10.18.1 DMA access

Assumptions

By making some assumptions, the bandwidth needed for each type of AHB transfer can be calculated and added in order to find the overall bandwidth requirement.

The flexibility of the descriptors used in the Ethernet block allows the possibility of defining memory buffers in a range of sizes. In order to analyze bus bandwidth requirements, some assumptions must be made about these buffers. The "worst case" is not addressed since that would involve all descriptors pointing to single byte buffers, with most of the memory occupied in holding descriptors and very little data. It can easily be shown that the AHB cannot handle the huge amount of bus traffic that would be caused by such a degenerate (and illogical) case.

For this analysis, an Ethernet packet is assumed to consist of a 64 byte frame. Continuous traffic is assumed on both the transmit and receive channels.

This analysis does not reflect the flow of Ethernet traffic over time, which would include inter-packet gaps in both the transmit and receive channels that reduce the bandwidth requirements over a larger time frame.

Types of DMA access and their bandwidth requirements

The interface to an external Ethernet PHY is via RMII. RMII operates at 50 MHz, transferring a byte in 4 clock cycles. The data transfer rate is 12.5 Mbps.

The Ethernet block initiates DMA accesses for the following cases:

- Tx descriptor read:
 - Transmit descriptors occupy 2 words (8 bytes) of memory and are read once for each use of a descriptor.
 - Two word read happens once every 64 bytes (16 words) of transmitted data.
 - This gives 1/8th of the data rate, which = 1.5625 Mbps.
- Rx descriptor read:
 - Receive descriptors occupy 2 words (8 bytes) of memory and are read once for each use of a descriptor.
 - Two word read happens once every 64 bytes (16 words) of received data.
 - This gives 1/8th of the data rate, which = 1.5625 Mbps.
- Tx status write:
 - Transmit status occupies 1 word (4 bytes) of memory and is written once for each use of a descriptor.
 - One word write happens once every 64 bytes (16 words) of transmitted data.
 - This gives 1/16th of the data rate, which = 0.7813 Mbps.

- Rx status write:
 - Receive status occupies 2 words (8 bytes) of memory and is written once for each use of a descriptor.
 - Two word write happens once every 64 bytes (16 words) of received data.
 - This gives 1/8 of the data rate, which = 1.5625 Mbps.
- Tx data read:
 - Data transmitted in an Ethernet frame, the size is variable.
 - Basic Ethernet rate = 12.5 Mbps.
- Rx data write:
 - Data to be received in an Ethernet frame, the size is variable.
 - Basic Ethernet rate = 12.5 Mbps.

This gives a total rate of 30.5 Mbps for the traffic generated by the Ethernet DMA function.

10.18.2 Types of CPU access

- Accesses that mirror each of the DMA access types:
 - All or part of status values must be read, and all or part of descriptors need to be written after each use, transmitted data must be stored in the memory by the CPU, and eventually received data must be retrieved from the memory by the CPU.
 - This gives roughly the same or slightly lower rate as the combined DMA functions, which = 30.5 Mbps.
- Access to registers in the Ethernet block:
 - The CPU must read the RxProduceIndex, TxConsumeIndex, and IntStatus registers, and both read and write the RxConsumeIndex and TxProduceIndex registers.
 - 7 word read/writes once every 64 bytes (16 words) of transmitted and received data.
 - This gives 7/16 of the data rate, which = 5.4688 Mbps.

This gives a total rate of 36 Mbps for the traffic generated by the Ethernet DMA function.

10.18.3 Overall bandwidth

Overall traffic on the AHB is the sum of DMA access rates and CPU access rates, which comes to approximately 66.5 MB/s.

The peak bandwidth requirement can be somewhat higher due to the use of small memory buffers, in order to hold often used addresses (e.g. the station address) for example. Driver software can determine how to build frames in an efficient manner that does not overutilize the AHB.

The bandwidth available on the AHB bus depends on the system clock frequency. As an example, assume that the system clock is set at 60 MHz. All or nearly all of bus accesses related to the Ethernet will be word transfers. The raw AHB bandwidth can be approximated as 4 bytes per two system clocks, which equals 2 times the system clock rate. With a 60 MHz system clock, the bandwidth is 120 MB/s, giving about 55% utilization

for Ethernet traffic during simultaneous transmit and receive operations. This shows that it is not necessary to use the maximum CPU frequency for the Ethernet to work with plenty of bandwidth headroom.

10.19 CRC calculation

The calculation is used for several purposes:

- Generation the FCS at the end of the Ethernet frame.
- Generation of the hash table index for the hash table filtering.
- Generation of the destination and source address hash CRCs.

The C pseudocode function below calculates the CRC on a frame taking the frame (without FCS) and the number of bytes in the frame as arguments. The function returns the CRC as a 32-bit integer.

```
int crc_calc(char frame_no_fcs[], int frame_len) {
    int i;    // iterator
    int j;    // another iterator
    char byte; // current byte
    int crc;  // CRC result
    int q0, q1, q2, q3; // temporary variables
    crc = 0xFFFFFFFF;
    for (i = 0; i < frame_len; i++) {
        byte = *frame_no_fcs++;
        for (j = 0; j < 2; j++) {
            if (((crc >> 28) ^ (byte >> 3)) & 0x00000001) {
                q3 = 0x04C11DB7;
            } else {
                q3 = 0x00000000;
            }
            if (((crc >> 29) ^ (byte >> 2)) & 0x00000001) {
                q2 = 0x09823B6E;
            } else {
                q2 = 0x00000000;
            }
            if (((crc >> 30) ^ (byte >> 1)) & 0x00000001) {
                q1 = 0x130476DC;
            } else {
                q1 = 0x00000000;
            }
            if (((crc >> 31) ^ (byte >> 0)) & 0x00000001) {
                q0 = 0x2608EDB8;
            } else {
                q0 = 0x00000000;
            }
            crc = (crc << 4) ^ q3 ^ q2 ^ q1 ^ q0;
            byte >>= 4;
        }
    }
    return crc;
}
```

For FCS calculation, this function is passed a pointer to the first byte of the frame and the length of the frame without the FCS.

For hash filtering, this function is passed a pointer to the destination address part of the frame and the CRC is only calculated on the 6 address bytes. The hash filter uses bits [28:23] for indexing the 64-bits { HashFilterH, HashFilterL } vector. If the corresponding bit is set the packet is passed, otherwise it is rejected by the hash filter.

For obtaining the destination and source address hash CRCs, this function calculates first both the 32-bit CRCs, then the nine most significant bits from each 32-bit CRC are extracted, concatenated, and written in every StatusHashCRC word of every fragment status.